
MAXIMIZING LENDING PROFIT WITH RISK- AWARE TARGETING (LENDING CLUB DATA)

Predicting default risk and
turning scores into dollars

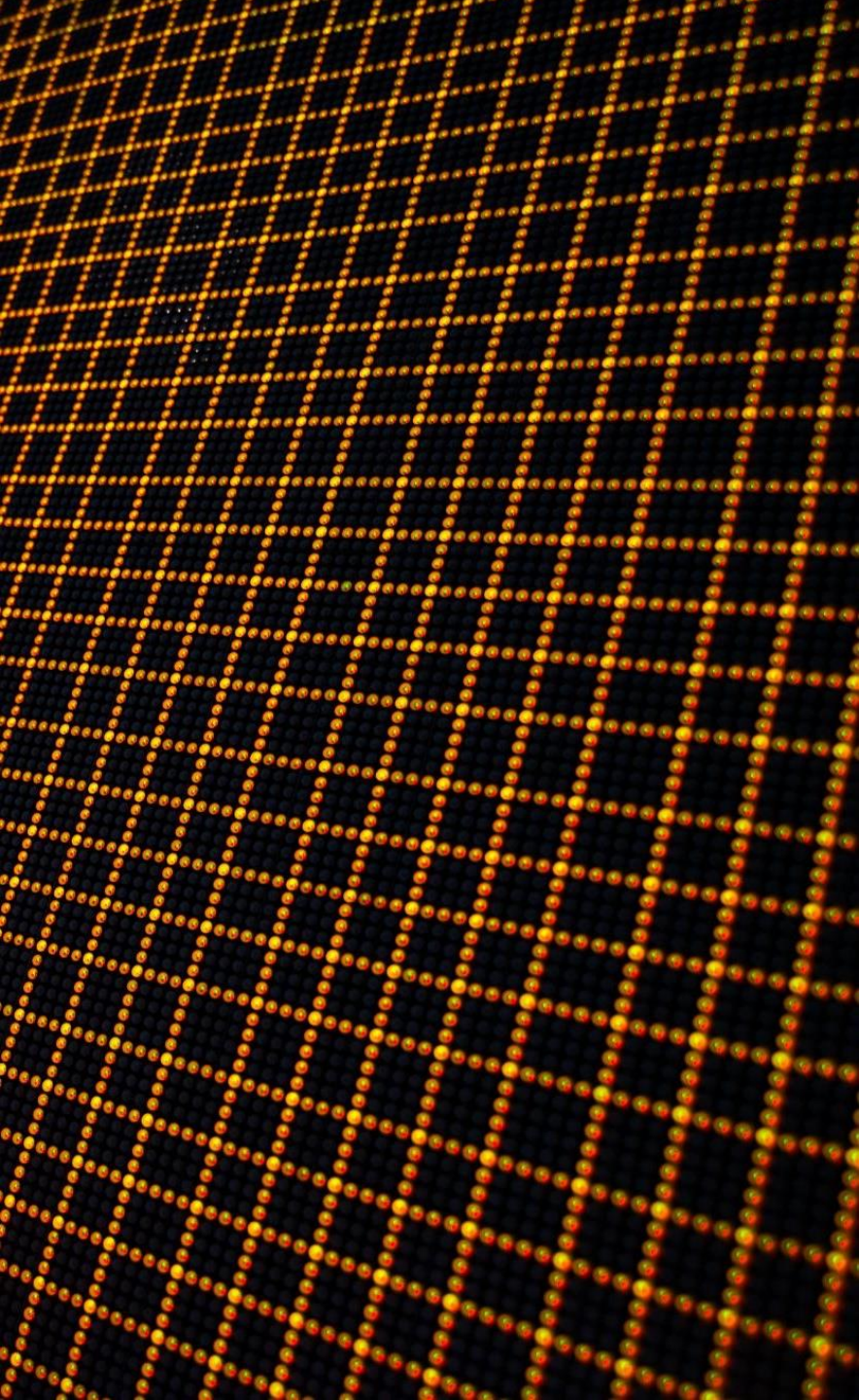
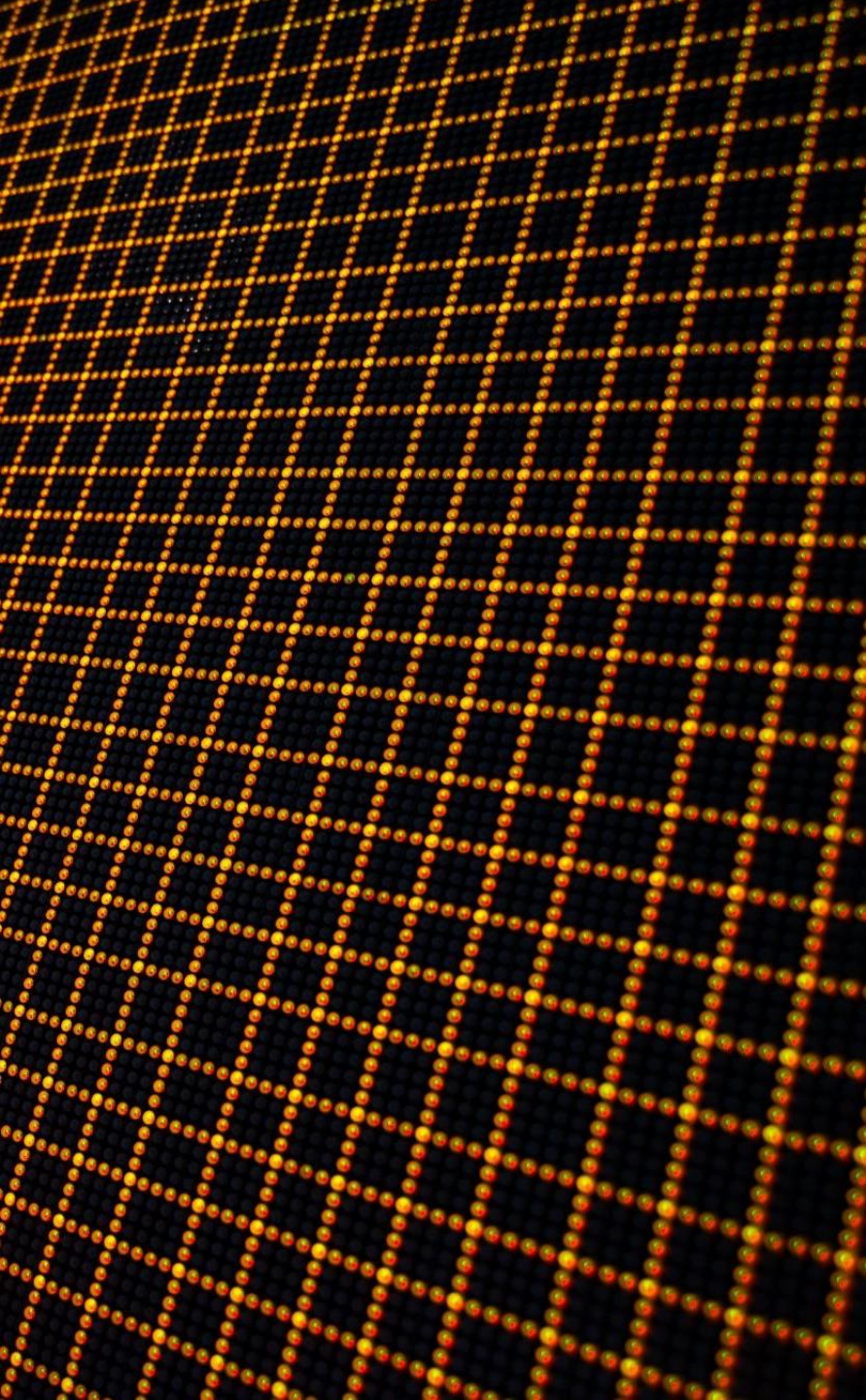


TABLE OF CONTENTS

- Introduction
 - Overview
 - Data
 - Methodology
 - Results
 - Conclusion
-



INTRODUCTION

The Problem

- Lending to the wrong customers exposes the business to **significant financial losses** through defaults.
- Traditional credit assessments often fail to accurately identify **high-risk individuals**.

The Opportunity

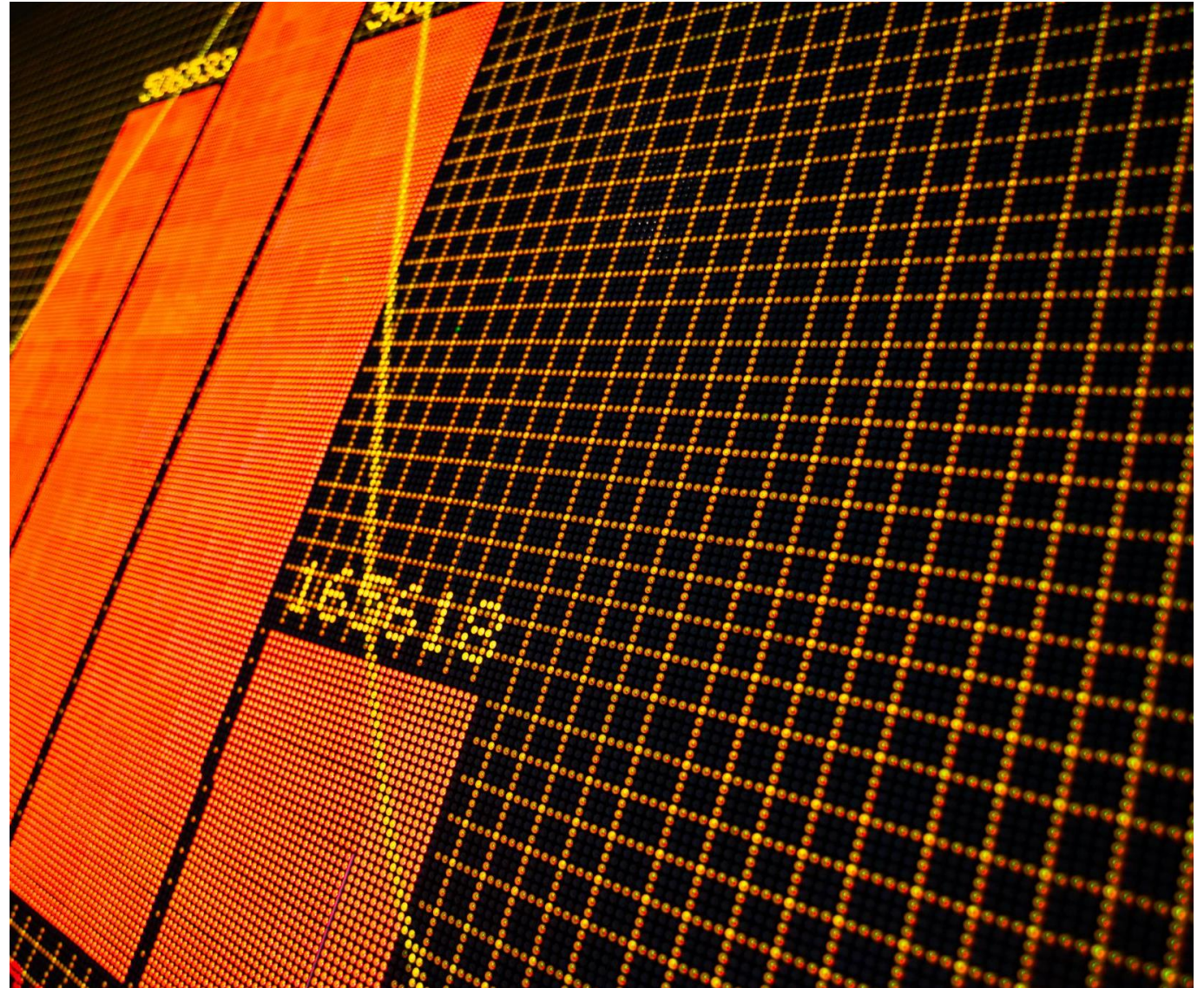
By leveraging predictive modeling, we can **differentiate between profitable customers and potential defaulters**.

This enables smarter decision-making:

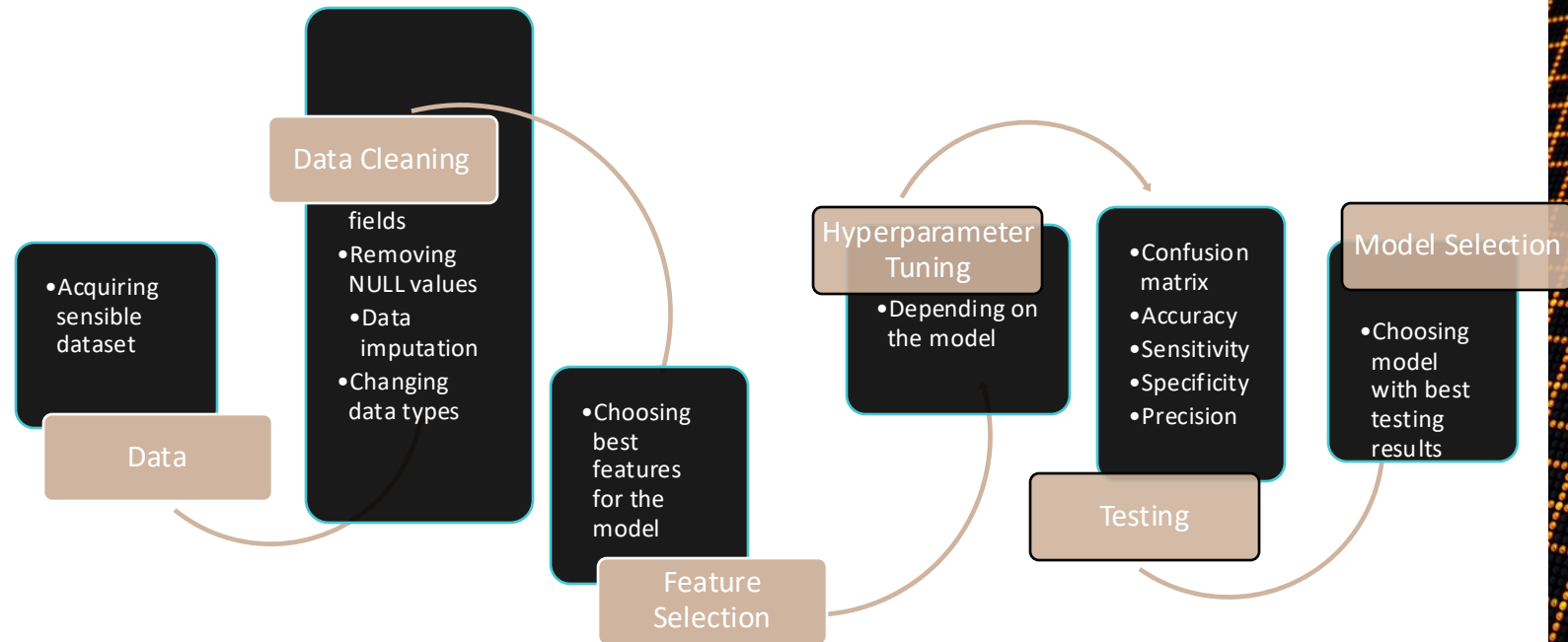
- Decline or adjust terms for **high-risk borrowers**.
 - Offer competitive products to **low-risk, profitable customers**.
 - **The Impact**
 - **Profit maximization** by reducing default losses.
 - **Risk-adjusted pricing strategies** that improve portfolio resilience.
 - **Data-driven targeting** that aligns credit decisions with business growth objectives.
-

OVERVIEW

- Dataset
 - Preliminary data analysis
- Default identification – customers that are most likely to default
- Choosing data science/ML methods to be implemented
 - Dividing dataset into train and test
 - Feature engineering – choosing the best features for each model
 - Choosing the best models



PROCESS



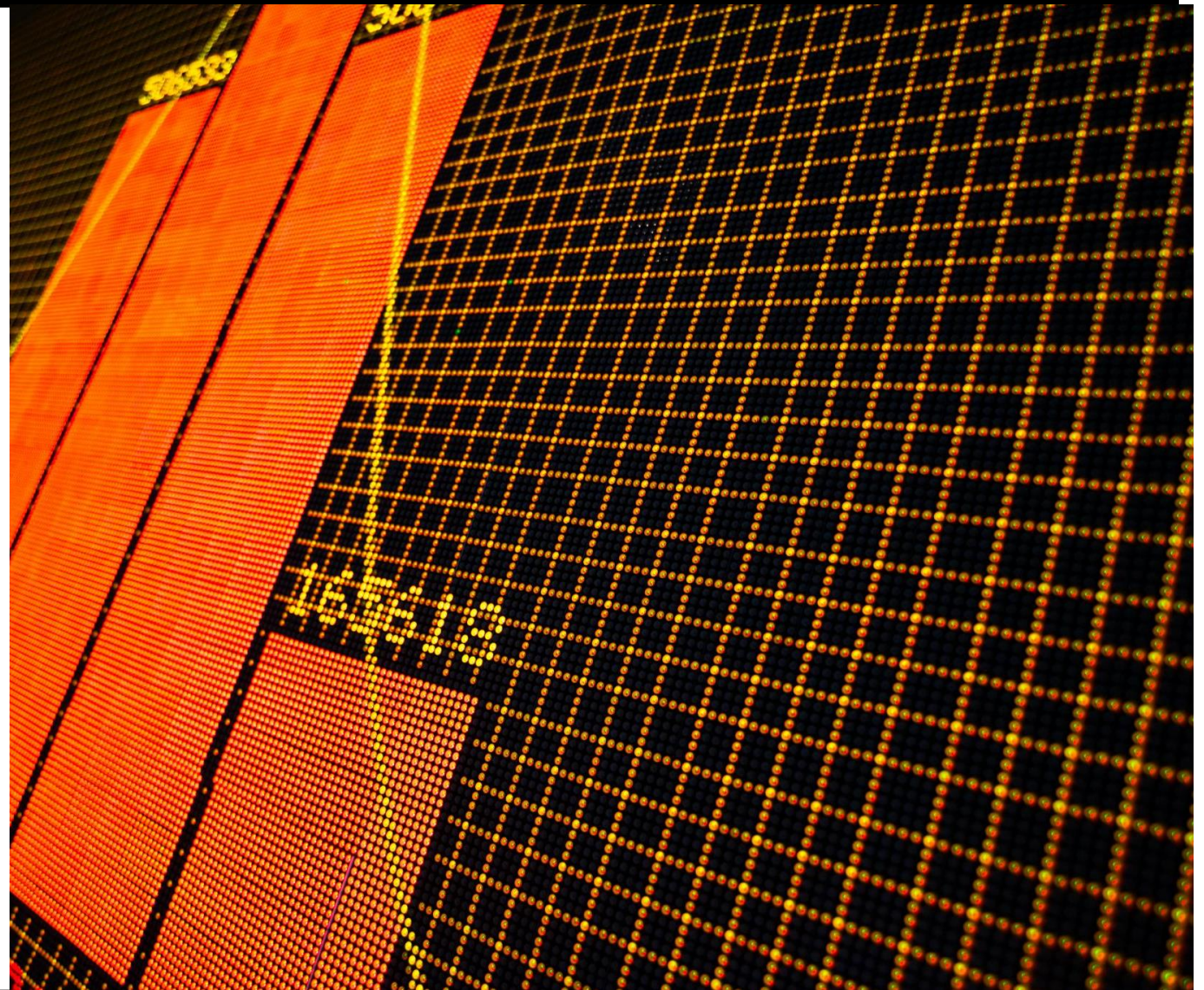
DATA

Original Data Set :

- 10.000 rows and 28 columns (7 categorical and 21 continuous)
- Default probability 13%

After transformation

- 9999 rows and 180 columns



DATA

- Missing values

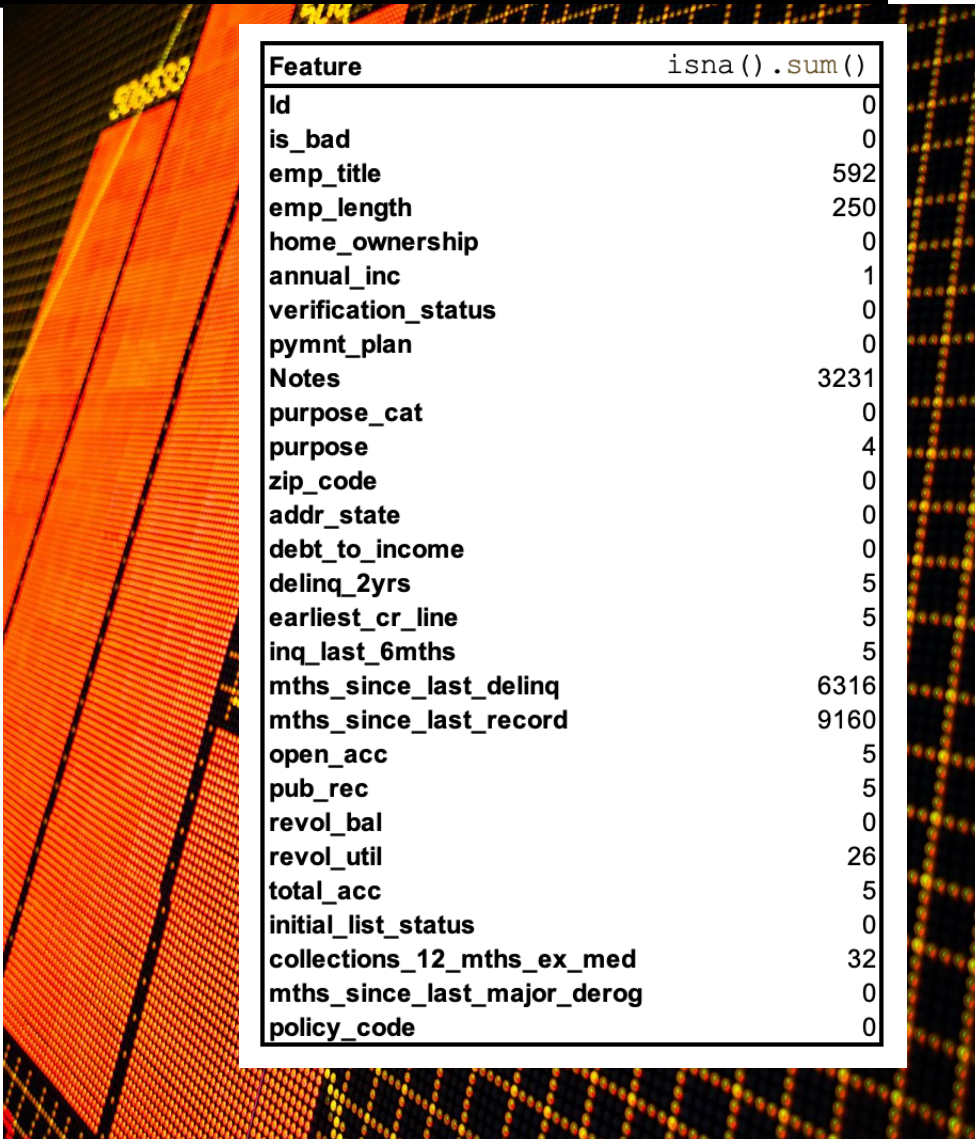
Feature	isna().sum()
Id	0
is_bad	0
emp_title	592
emp_length	250
home_ownership	0
annual_inc	1
verification_status	0
pymnt_plan	0
Notes	3231
purpose_cat	0
purpose	4
zip_code	0
addr_state	0
debt_to_income	0
delinq_2yrs	5
earliest_cr_line	5
inq_last_6mths	5
mths_since_last_delinq	6316
mths_since_last_record	9160
open_acc	5
pub_rec	5
revol_bal	0
revol_util	26
total_acc	5
initial_list_status	0
collections_12_mths_ex_med	32
mths_since_last_major_derog	0
policy_code	0

MISSING DATA

We can divide features with missing values into a few groups:

- Features that most likely won't bring much into our model, like *emp_title* or *Notes*, so I won't bother with replacing those missing values
- Features that don't have NaN (missing values) but have a different values instead (like na) for example *emp_length* - impute the median
- Features that have high percentage of NaN (*mths_since_last_delinq* or *mths_since_last_record*) - will address it further
- Features that need more investigation for the best imputation methods - either numbers (mean or median) - *delinq_2yrs*, *pub_rec*, *revol_util*, *total_acc*, *collections_12_mths_ex_med*
- Features that have only a few observations missing (0.05 % of the data - *purpose*, *delinq_2yrs*, *earliest_cr_line*, *inq_last_6mths*, *total_acc*)

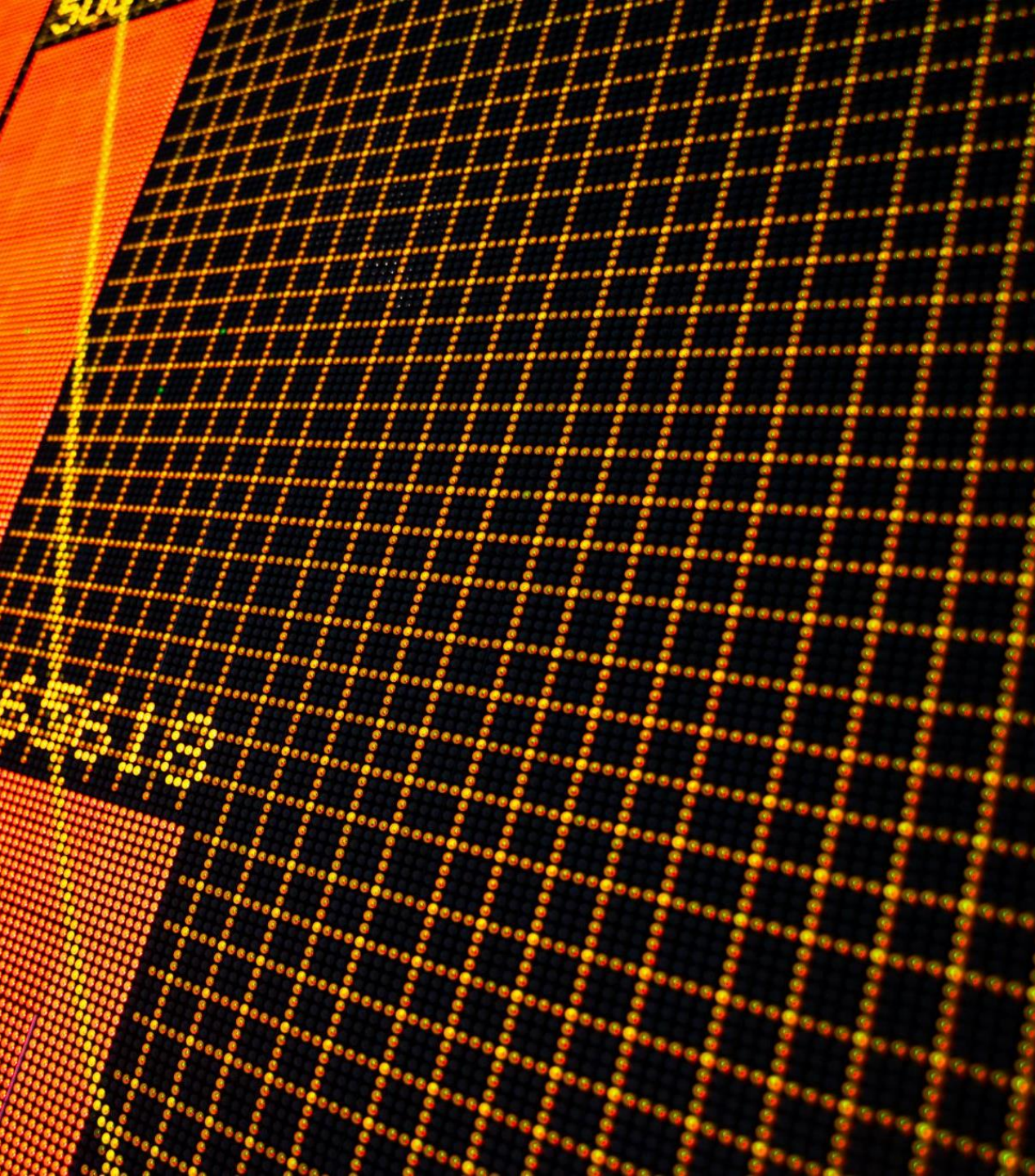
Important thing to remember when imputing anything else than 0, like mean or median is that it should be mean or median from the test or train dataset appropriately. For simplicity I am going to use median from the whole data set, however I am risking a data leakage in this case.



Feature	isna().sum()
Id	0
is_bad	0
emp_title	592
emp_length	250
home_ownership	0
annual_inc	1
verification_status	0
pymnt_plan	0
Notes	3231
purpose_cat	0
purpose	4
zip_code	0
addr_state	0
debt_to_income	0
delinq_2yrs	5
earliest_cr_line	5
inq_last_6mths	5
mths_since_last_delinq	6316
mths_since_last_record	9160
open_acc	5
pub_rec	5
revol_bal	0
revol_util	26
total_acc	5
initial_list_status	0
collections_12_mths_ex_med	32
mths_since_last_major_derog	0
policy_code	0

DATA DESCRIPTIVE STATISTICS

	count	mean	std	min	25%	50%	75%	max
delinq_2yrs	9995	0	1	0	0	0	0	11
inq_last_6mths	9995	1	1	0	0	1	2	25
mths_since_last_delinq	3684	36	22	0	18	34	53	120
mths_since_last_record	840	62	46	0	0	86	101	119
open_acc	9995	9	5	1	6	9	12	39
pub_rec	9995	0	0	0	0	0	0	3
revol_util	9974	48	28	0	25	49	72	101
total_acc	9995	22	12	1	13	20	29	90
collections_12_mths_ex_med	9968	0	0	0	0	0	0	0
mths_since_last_delinq_capped	3684	36	22	0	18	34	53	120
mths_since_last_record_capped	840	62	46	0	0	86	101	119



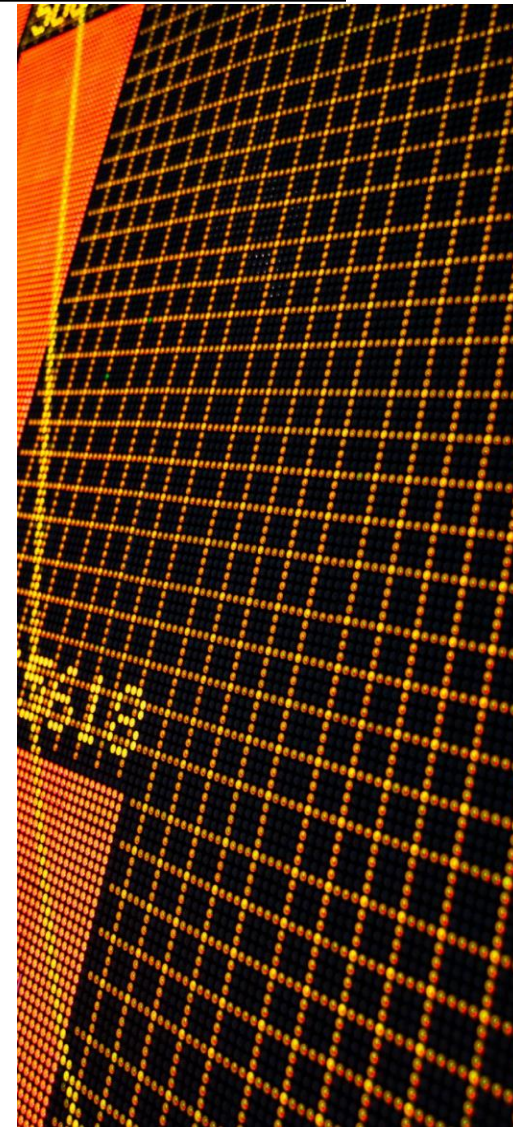
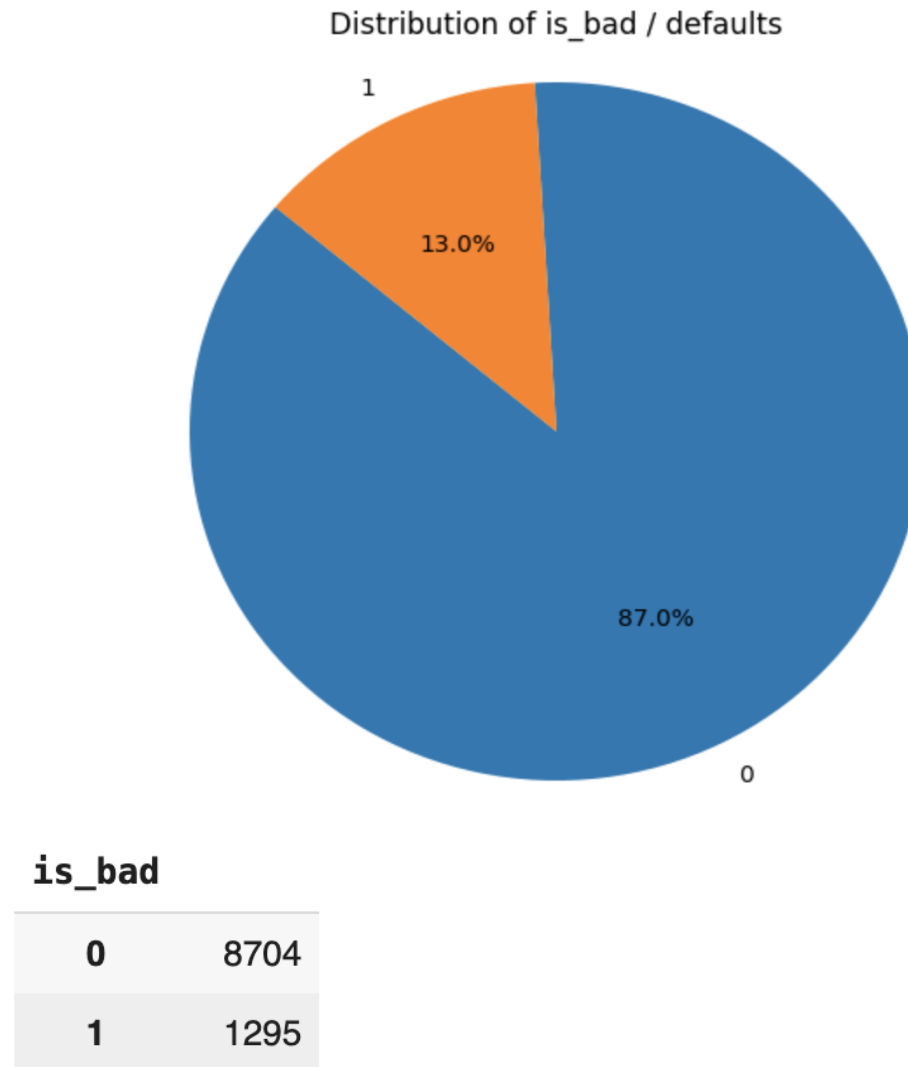
DATA – FEATURES

Interesting feature's properties:

- *collections_12_mths_ex_med* = all 0 with some missing values, dropping
 - *verification_status* can be a boolean 0 and 1 variables
 - *pymnt_plan* can be a boolean 0 and 1 variables
 - *purpose_cat* can be divided into 2 features: *purpose_cat* (categorical) and *small_business* (boolean)
 - *debt_to_income* can be divided into brackets, but for now I'll leave it as a numerical variable
 - *policy_code* could be converted into either numerical variable if there is an order in policy code PC1, PC2 or kept as categorical variables (that will need one hot encoding) if there's no particular order.
 - *zip_code* should also be converted - will focus on this later as
-

EXPLORATORY DATA ANALYSIS (EDA)

- We can see from the pie chart that the data set is **unbalanced** -> out of 10000 people, only ~ 13% are "bad", so only 13% have a high probability of default.
- For the modeling purpose, I would highly suggest creating another data set that's balanced-> 50% of it would be Yes (or 1) for ***is_bad***.
- By checking the distribution of ***is_bad*** -> **baseline accuracy** - anything lower than 87.1% is less than guessing just from the data (without the model we can assume that 87.1 % of candidates won't default).

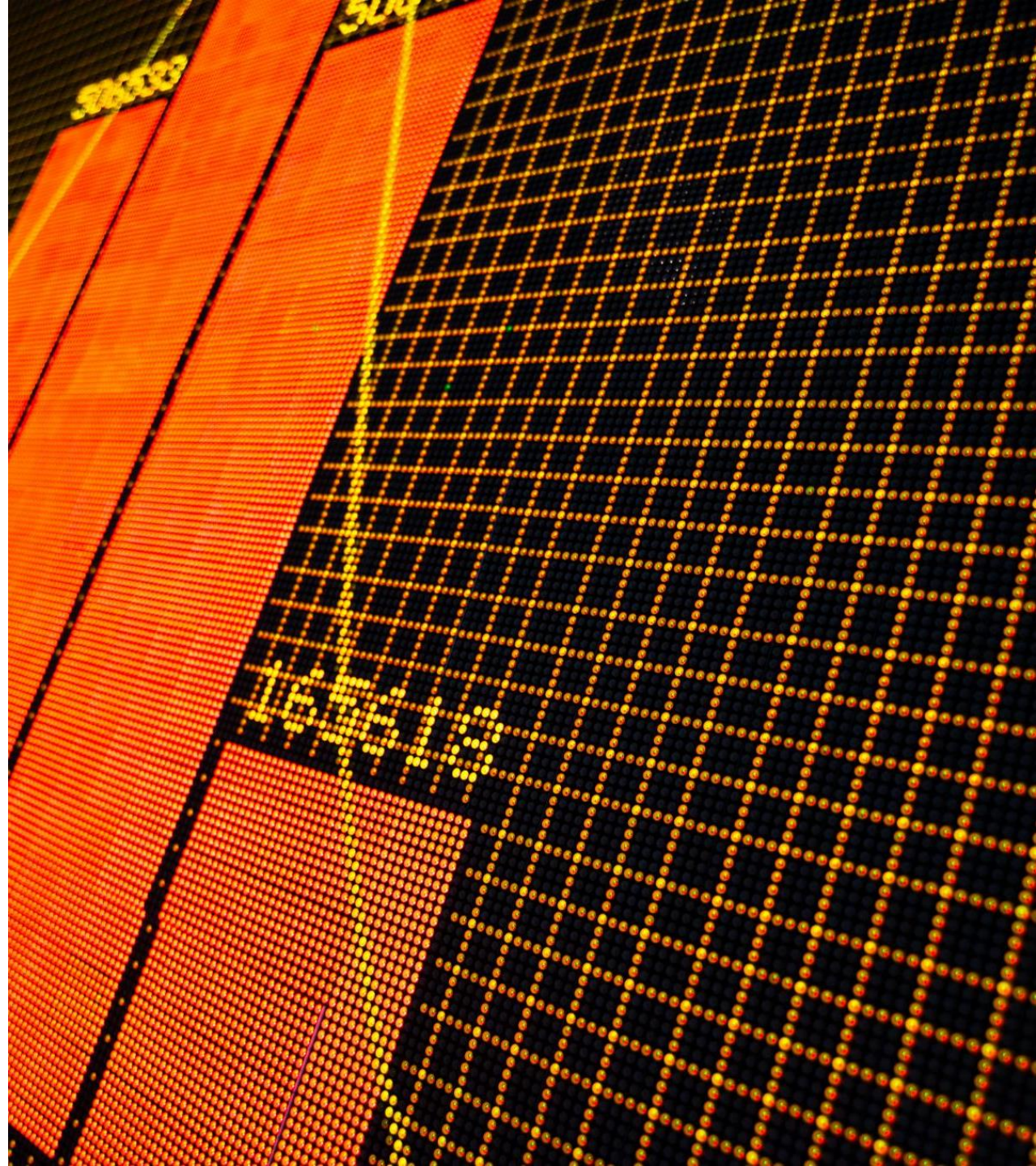


DATA - CORRELATIONS

nr	feature	measure	score	p_value	kind	rank_score
84	purpose_cat	Cramér's V	0.368	NaN	cat↔cat	0.3676
70	small_business	Spearman ρ	0.263	3.00E-158	num↔cat	0.2633
57	revol_util_imp	Point-biserial r_pb	0.088	1.37E-18	num↔bin	0.0878
60	revol_util_frac	Point-biserial r_pb	0.088	1.37E-18	num↔bin	0.0878
3	emp_length	Cramér's V	0.081	NaN	cat↔cat	0.0808
6	annual_inc	Spearman ρ	-0.072	5.16E-13	num↔cat	0.0721
72	zip3	Cramér's V	0.064	NaN	cat↔cat	0.0641
12	zip_code	Cramér's V	0.064	NaN	cat↔cat	0.0641
55	total_acc_imp	Spearman ρ	-0.063	2.67E-10	num↔cat	0.0631
14	addr_state	Cramér's V	0.054	NaN	cat↔cat	0.0542
8	verification_status	Cramér's V	0.051	NaN	cat↔cat	0.0507
64	verification_status_ind	Spearman ρ	0.050	5.33E-07	num↔cat	0.0501
67	pymnt_plan_ind	Spearman ρ	0.037	2.45E-04	num↔cat	0.0367
52	pub_rec_imp	Spearman ρ	0.034	5.98E-04	num↔cat	0.0343
46	inq_last_6mths_imp	Spearman ρ	0.034	6.78E-04	num↔cat	0.0340
17	debt_to_income	Spearman ρ	0.031	2.12E-03	num↔cat	0.0307
40	emp_length_num	Spearman ρ	-0.030	2.42E-03	num↔cat	0.0303
80	home_ownership	Cramér's V	0.027	NaN	cat↔cat	0.0269
10	pymnt_plan	Cramér's V	0.024	NaN	cat↔cat	0.0242
82	policy_code	Cramér's V	0.022	NaN	cat↔cat	0.0222

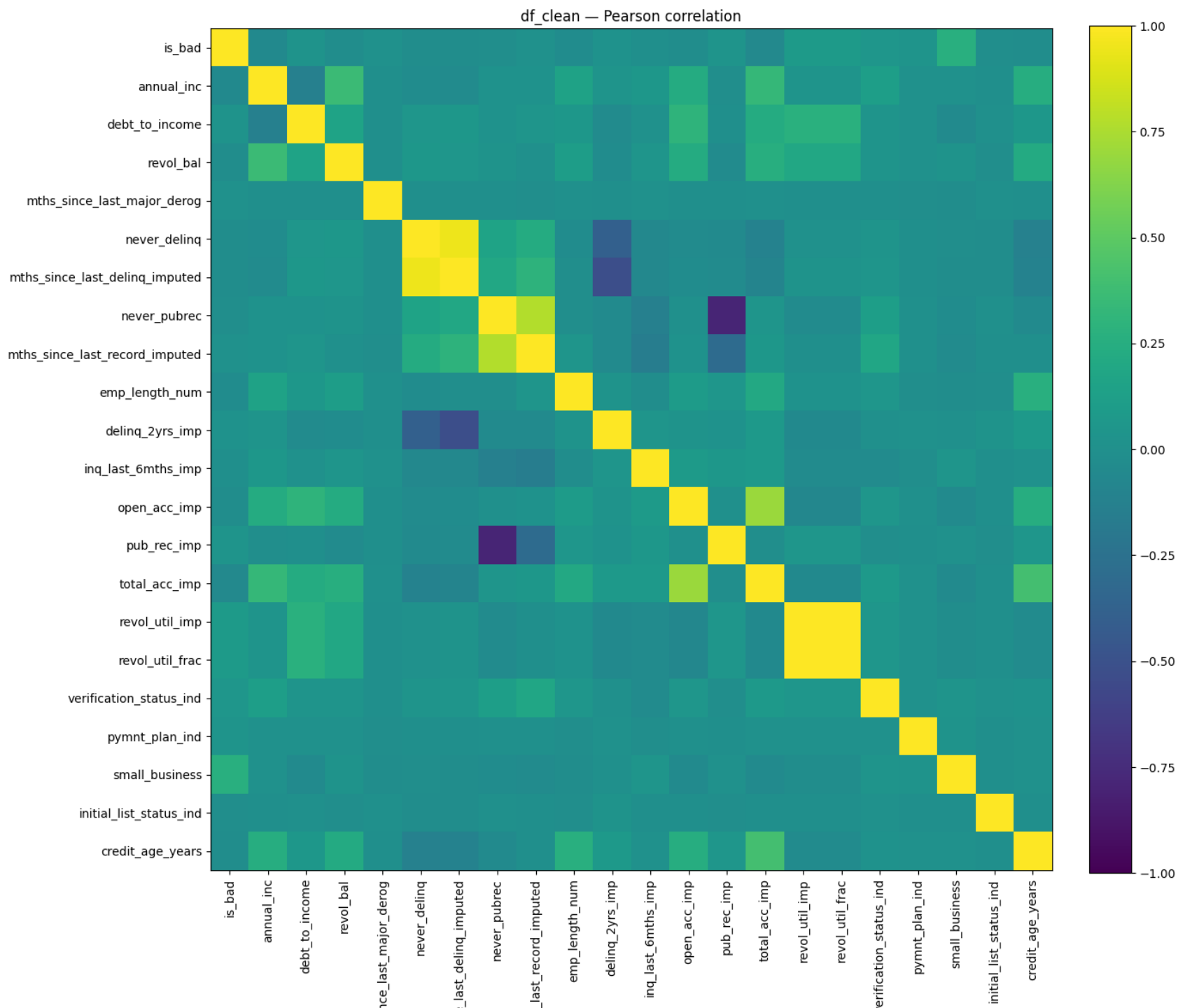
DATA - CORRELATIONS

- **feature** — the predictor being tested against your target (is_bad).
- **pearson_r** — Pearson correlation coefficient (linear association)
 - range [-1, 1]
 - **Sign:** + means as the feature increases, is_bad tends to increase; - means the opposite.
 - **Magnitude:** $|r|$ close to 0 = weak; ~ 0.1 small, ~ 0.3 medium, $\sim 0.5+$ large (rough Cohen-style rule of thumb).
- **pearson_p** — p-value for the null hypothesis $r = 0$ (two-sided). Very small p means the observed r is unlikely under “no linear association.” With large n, even tiny r can be “statistically significant.”
- **spearman_rho** — Spearman rank correlation (monotonic association), [-1, 1]. Less sensitive to outliers and linearity; compares ranks instead of raw values.
- **spearman_p** — p-value for the null hypothesis $\rho = 0$ (two-sided).
- **n** — number of rows used in the calculation (after joint NaN filtering).
- **note** — diagnostic flag (e.g., “non-numeric”, “constant/insufficient”, etc.). Blank means no issue.



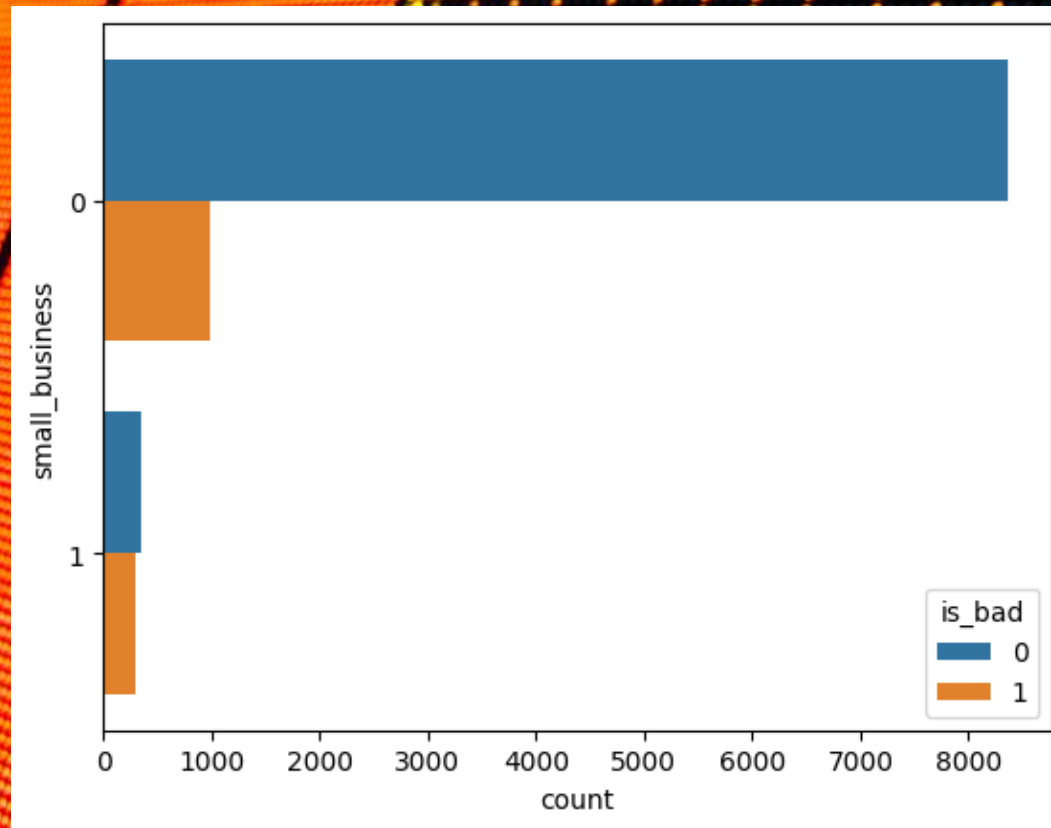
DATA - CORRELATIONS

nr	feature	pearson_r	pearson_p	spearman_rho	spearman_p	n
18	small_business	0.263	3.0E-158	2.6E-01	3.0E-158	9999
14	revol_util_imp	0.088	1.4E-18	8.8E-02	1.4E-18	9999
15	revol_util_frac	0.088	1.4E-18	8.8E-02	1.4E-18	9999
16	verification_status_ind	0.050	5.3E-07	5.0E-02	5.3E-07	9999
17	pymnt_plan_ind	0.037	2.4E-04	3.7E-02	2.4E-04	9999
12	pub_rec_imp	0.032	1.3E-03	3.4E-02	6.0E-04	9999
1	debt_to_income	0.030	2.9E-03	3.1E-02	2.1E-03	9999
9	delinq_2yrs_imp	0.022	2.9E-02	1.7E-02	9.4E-02	9999
3	mths_since_last_major_derog	0.015	1.2E-01	1.5E-02	1.2E-01	9999
7	mths_since_last_record_imputed	0.015	1.4E-01	-1.3E-02	1.8E-01	9999
10	inq_last_6mths_imp	-0.001	9.2E-01	3.4E-02	6.8E-04	9999
19	initial_list_status_ind	-0.009	3.9E-01	-8.7E-03	3.9E-01	9999
6	never_pubrec	-0.015	1.3E-01	-1.5E-02	1.3E-01	9999
2	revol_bal	-0.016	1.0E-01	1.0E-02	3.2E-01	9999
5	mths_since_last_delinq_imputed	-0.017	9.6E-02	-1.7E-02	9.1E-02	9999
20	credit_age_years	-0.017	9.2E-02	-2.0E-02	4.9E-02	9999
4	never_delinq	-0.018	7.5E-02	-1.8E-02	7.5E-02	9999
11	open_acc_imp	-0.020	4.2E-02	-1.9E-02	5.6E-02	9999
8	emp_length_num	-0.024	1.6E-02	-3.0E-02	2.4E-03	9999
0	annual_inc	-0.051	3.4E-07	-7.2E-02	5.2E-13	9999
13	total_acc_imp	-0.055	3.2E-08	-6.3E-02	2.7E-10	9999

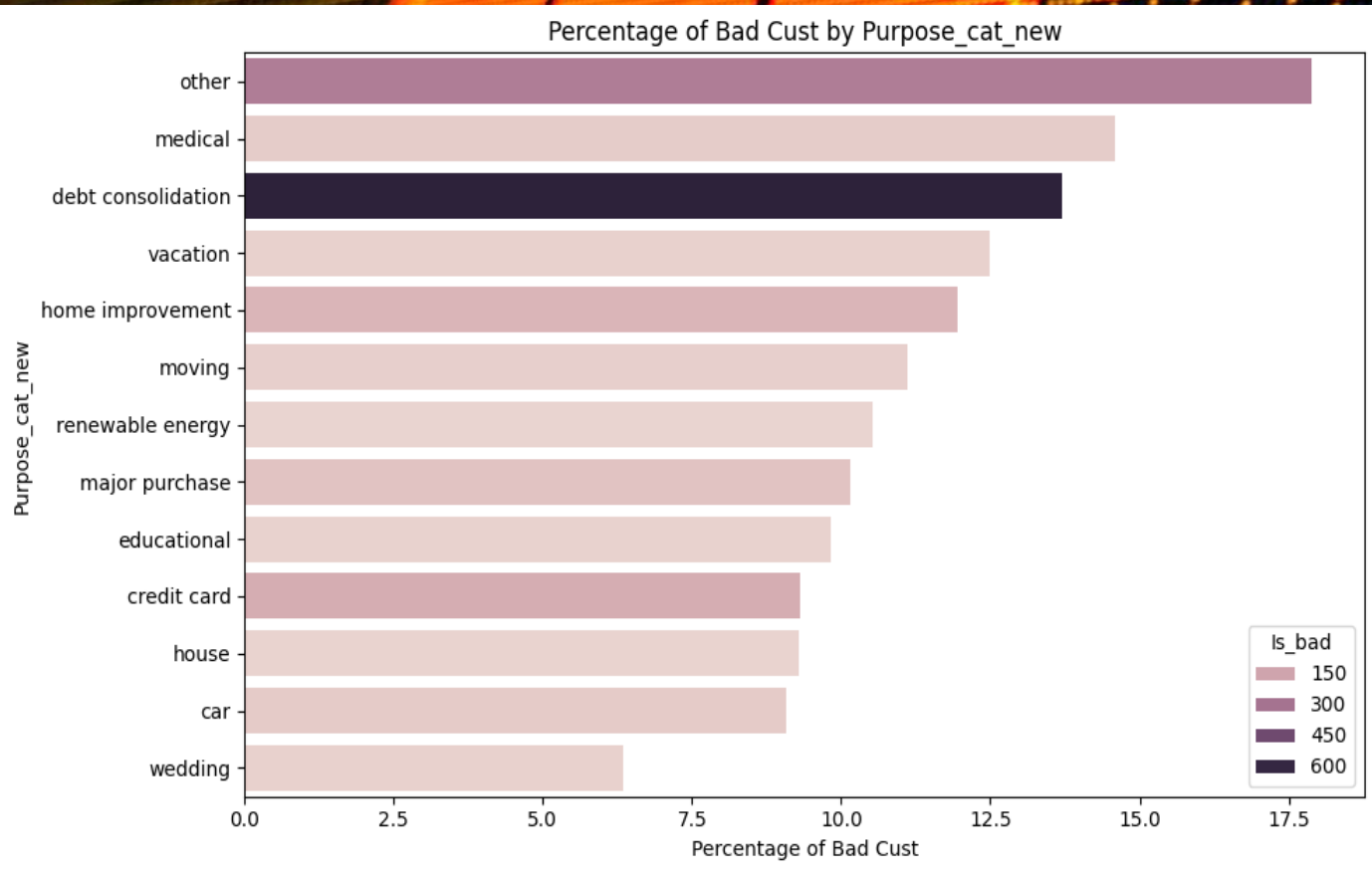


FEATURES - CORRELATION
MATRIX

FEARURES – SMALL_BUSINESS

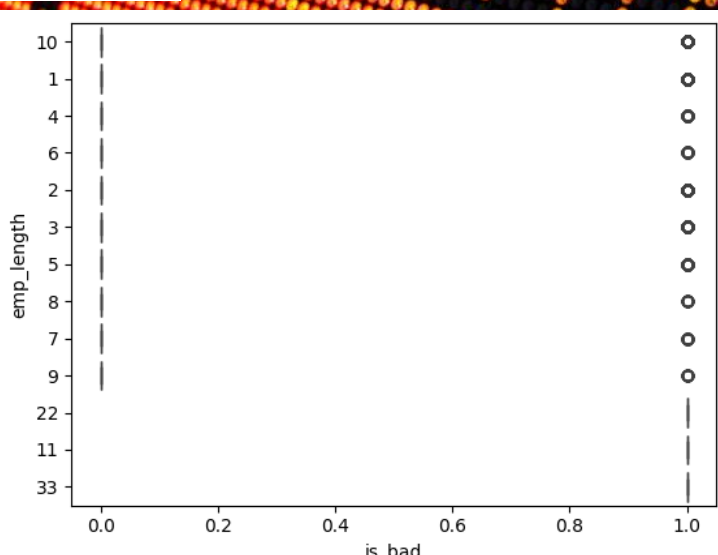
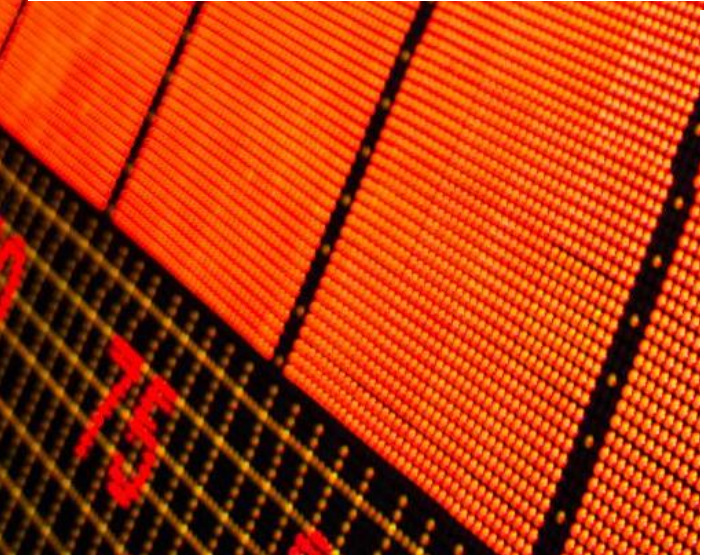
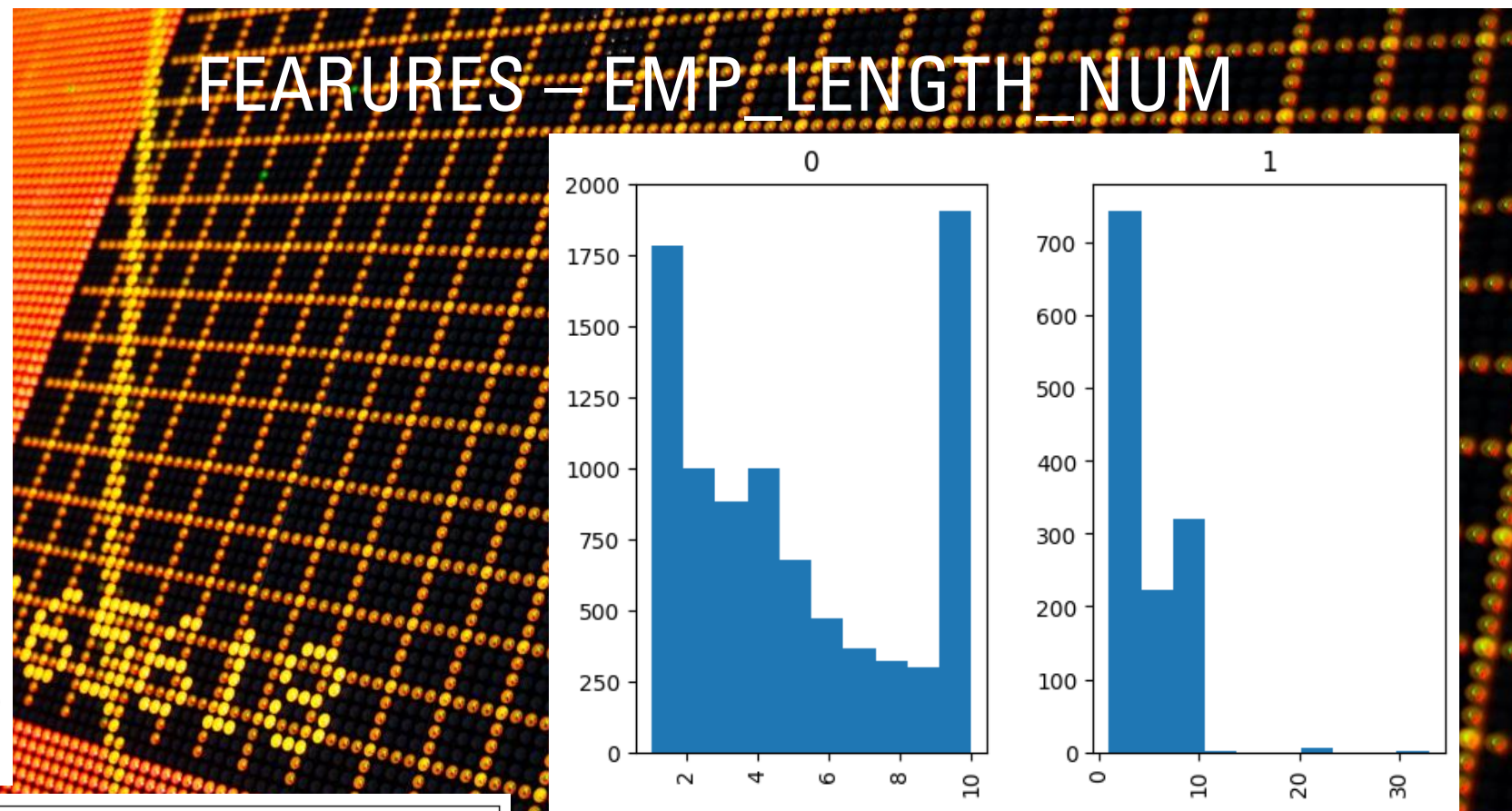
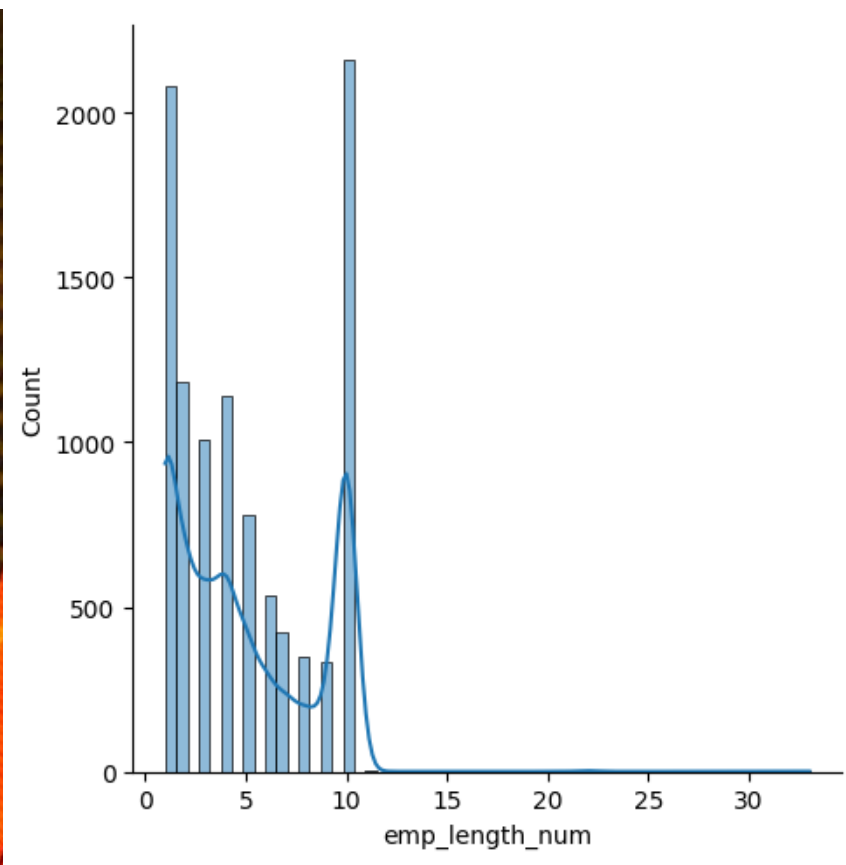


	Feature_class	Total_Cust	Is_bad	Percentage_bad
1	1	646	301	47
0	0	9353	994	11

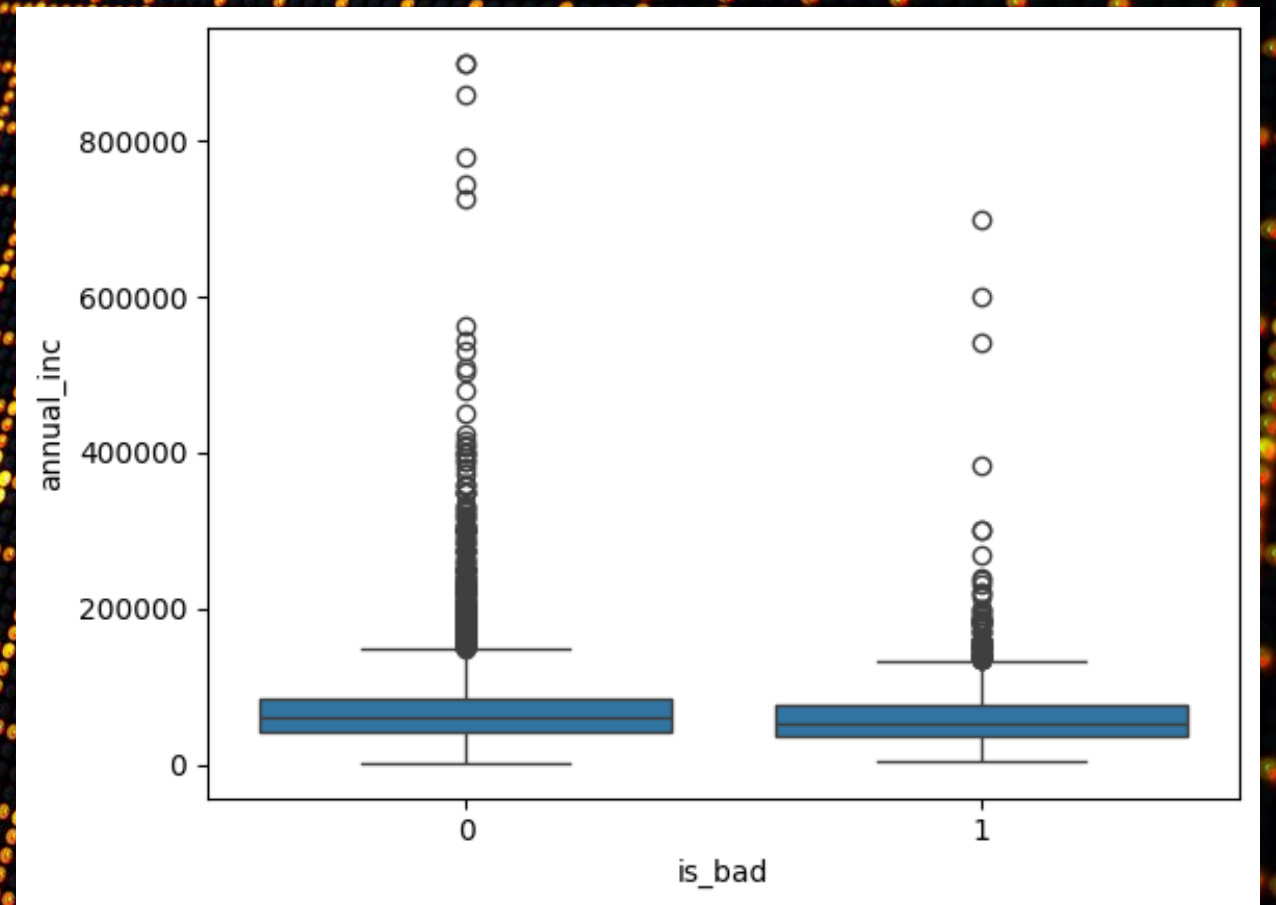
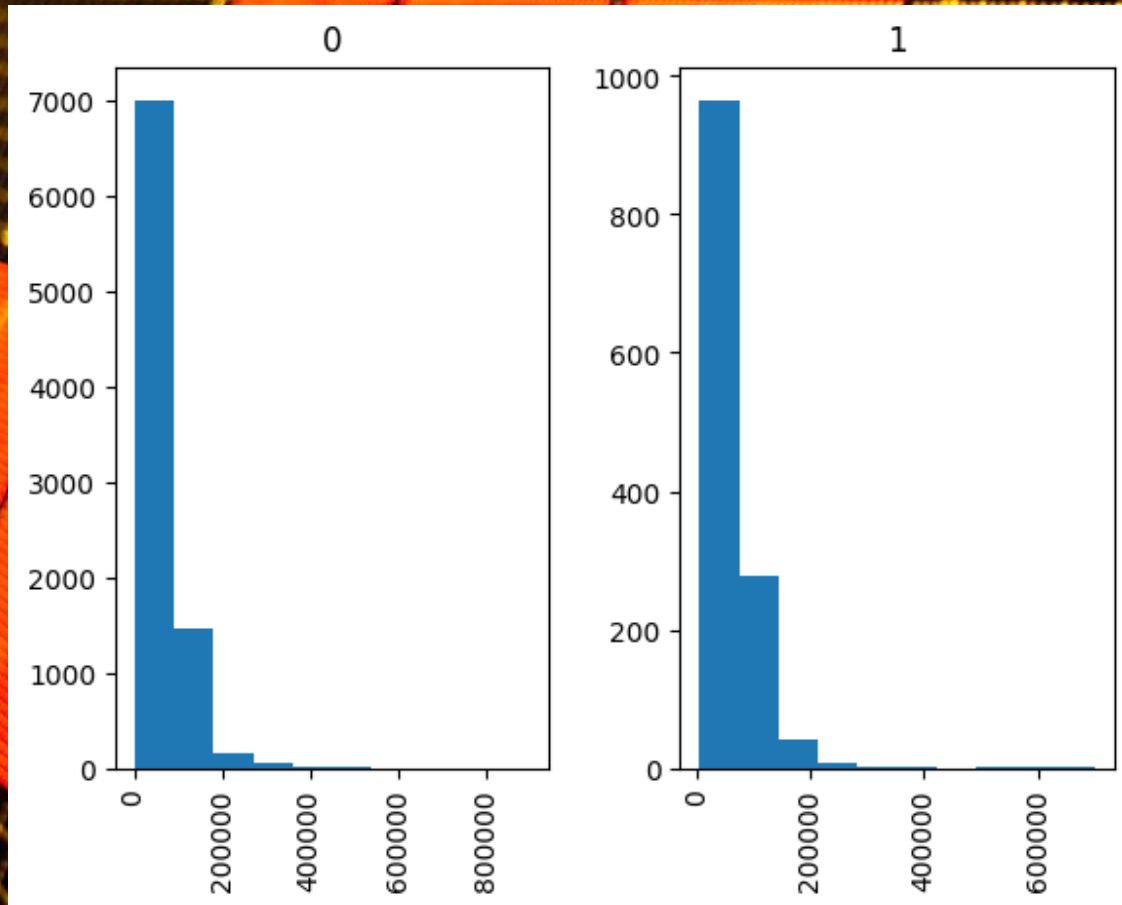


FEARURES – PURPOSE_CAT_NEW

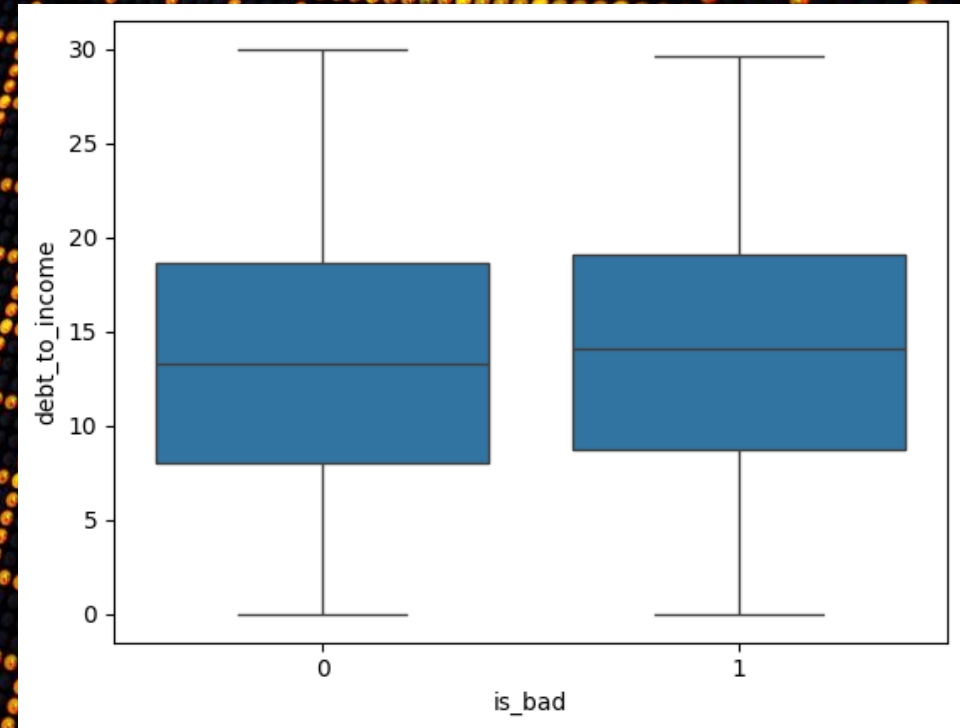
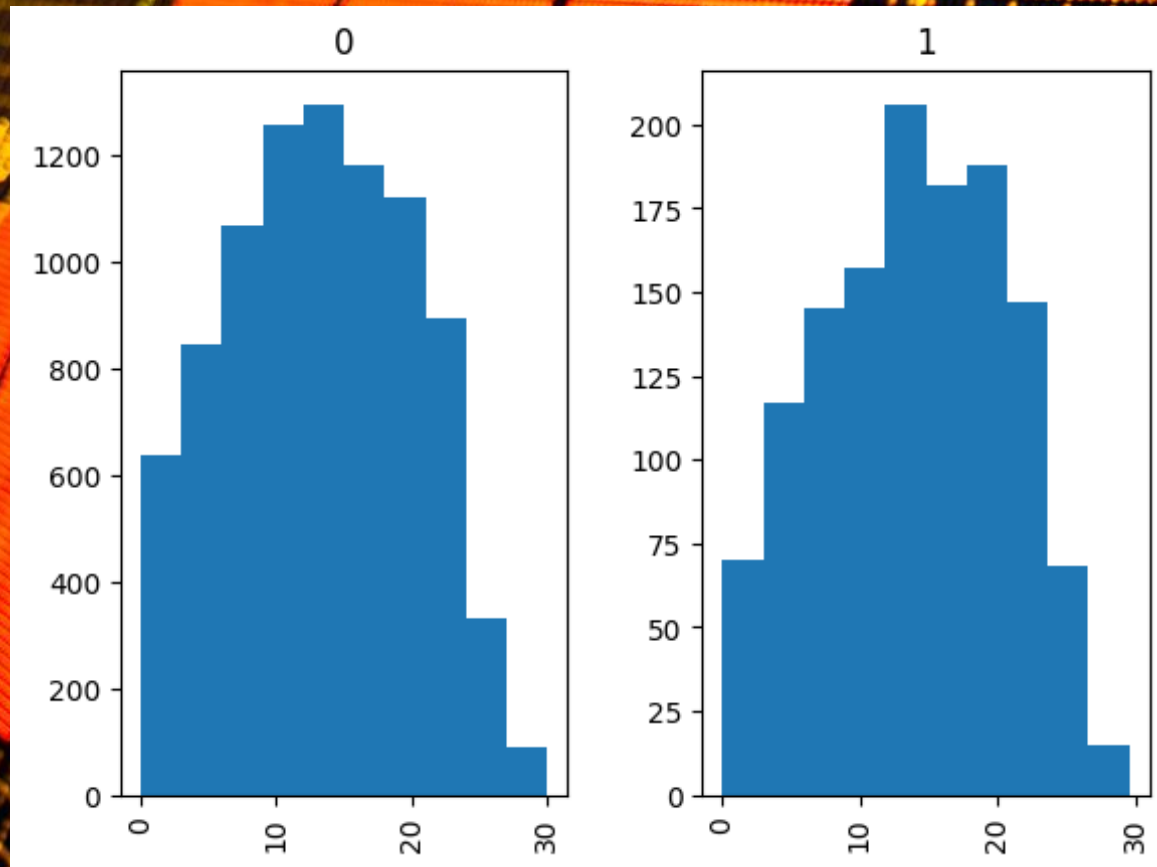
	Feature_class	Total_Cust	Is_bad	Percentage_bad
9	other	1516	271	18
7	medical	185	27	15
2	debt consolidation	4556	624	14
11	vacation	96	12	12
4	home improvement	811	97	12
8	moving	162	18	11
10	renewable energy	19	2	11
6	major purchase	551	56	10
3	educational	112	11	10
1	credit card	1290	120	9
5	house	97	9	9
0	car	352	32	9
12	wedding	252	16	6



FEARURES – ANNUAL_INC

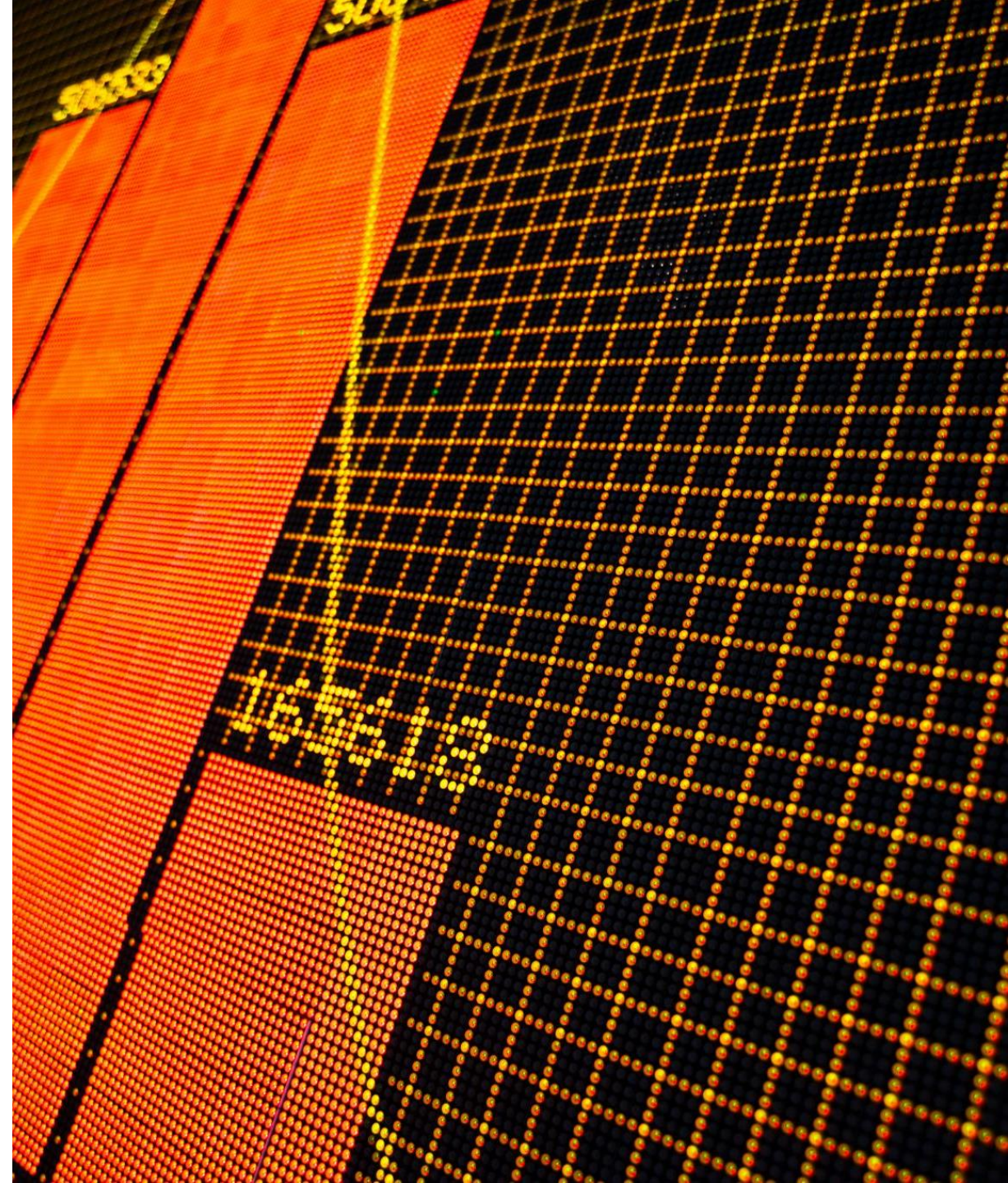


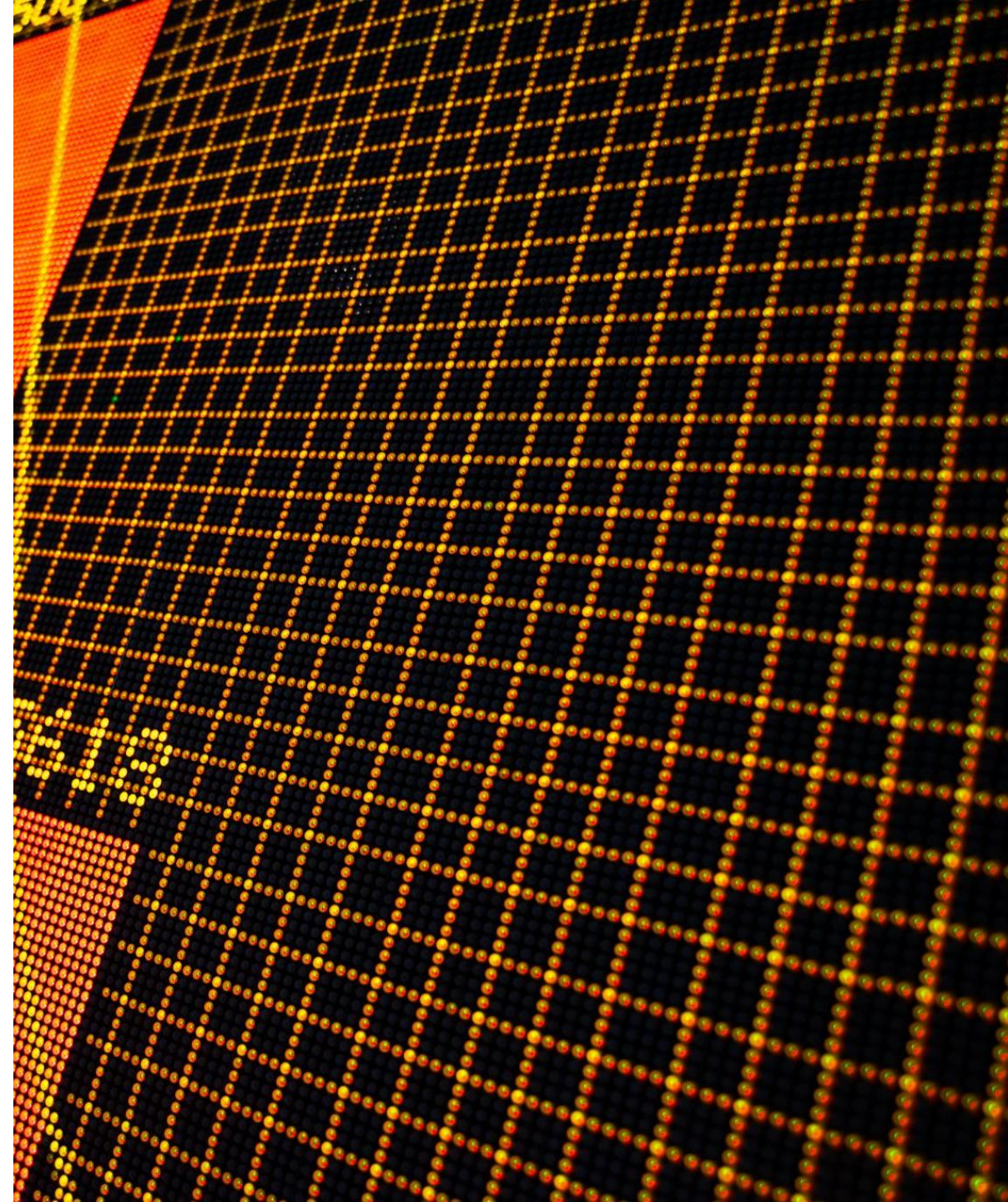
FEARURES – DEBT_TO_INCOME



METHODOLOGY

- Preliminary data analysis
- Explanatory Data Analysis
- Feature engineering
- Fitting model
- Choosing the best model
- Hyper parameters tuning
- Testing Model
- Model Deployment





MODELS

- Logistic Regression
- Decision Trees
- Random Forest
- Gradient Boosting
- Support Vector Machine
- Naïve Bayes
- K Nearest Neighbor

MODELS

Baseline – Logistic Regression

- Interpretable benchmark model
- Provides clear coefficients and odds ratios
- Serves as a reference point for more complex models

Decision Tree

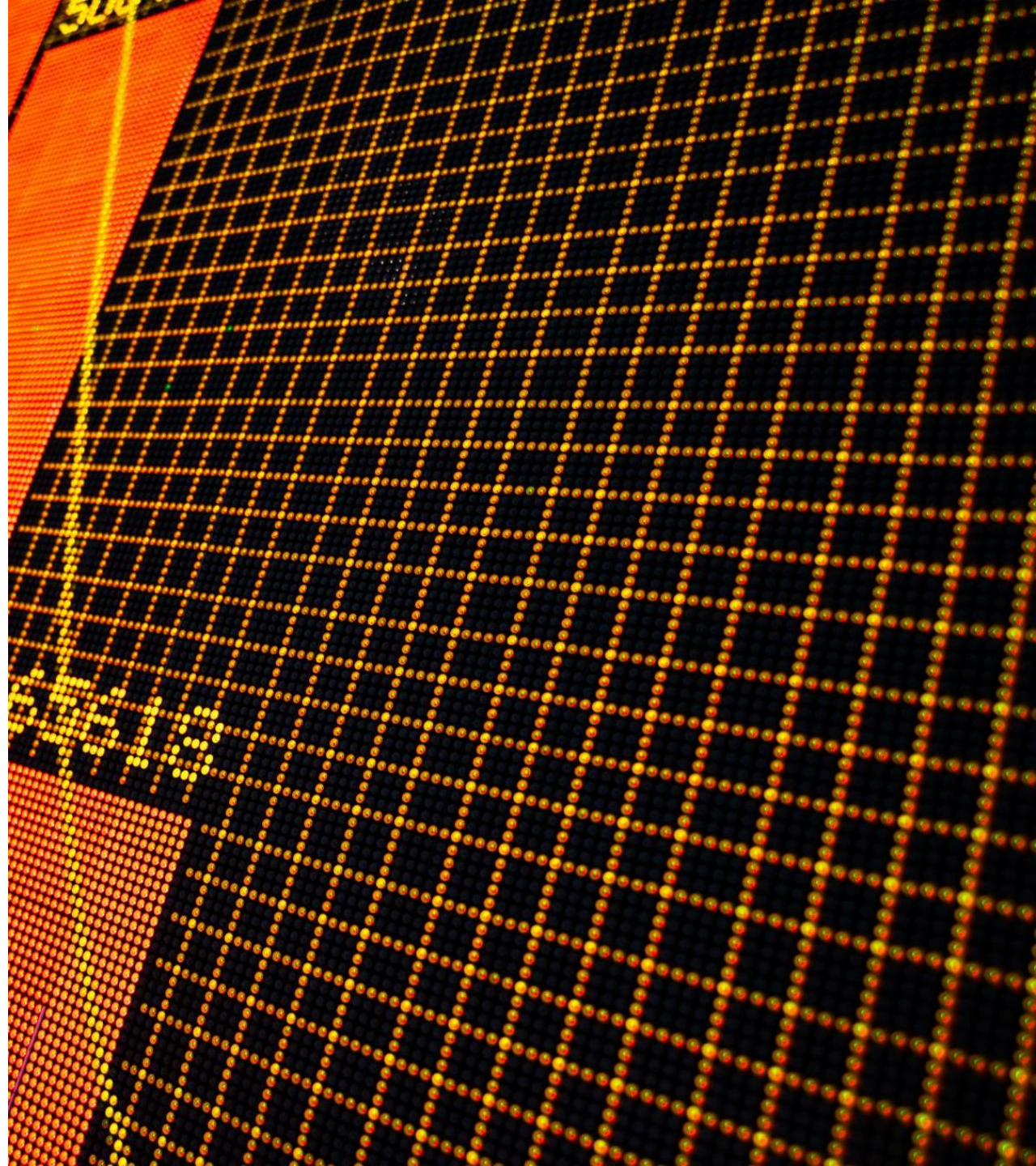
- Simple rules → easy to understand
- Mimics how underwriters make decisions (if/then rules)
- Limitation: can overfit and be unstable

Ensembles (Random Forest & others)

- Built on many decision trees → higher accuracy & robustness
- Reduce overfitting by averaging across trees
- Provide feature importance for interpretability

Model Selection Criteria

- Compared using validation metrics (AUC, precision, recall → f1)
 - Evaluated in terms of **business profit impact**
 - Balanced explainability (tree, logistic) with accuracy (ensemble)
-



BEST MODELS

- Balanced]data
- Ensemble models
 - XG Boost
 - Gradient Boosting
 - Random Forest
- Decision Tree

Nr	Model	Accuracy	Precision	Recall	F1 Score	ROC AUC	Source
15	XGBClassifier	0.886	0.679	0.207	0.317	0.596	Balanced
7	XGBClassifier	0.887	0.718	0.199	0.312	0.594	Original
23	XGBClassifier	0.887	0.718	0.199	0.312	0.594	Scaled
31	XGBClassifier	0.881	0.612	0.203	0.305	0.592	Scaled & Balanced
3	GradientBoostingClassifier	0.893	0.957	0.176	0.297	0.587	Original
19	GradientBoostingClassifier	0.893	0.957	0.176	0.297	0.587	Scaled
27	GradientBoostingClassifier	0.888	0.770	0.184	0.297	0.588	Scaled & Balanced
17	DecisionTreeClassifier	0.817	0.292	0.301	0.296	0.597	Scaled
28	SVC	0.743	0.223	0.406	0.288	0.599	Scaled & Balanced
1	DecisionTreeClassifier	0.812	0.278	0.293	0.285	0.591	Original
24	LogisticRegression	0.651	0.192	0.539	0.284	0.604	Scaled & Balanced
11	GradientBoostingClassifier	0.891	0.933	0.164	0.279	0.581	Balanced
16	LogisticRegression	0.885	0.745	0.160	0.264	0.576	Scaled
9	DecisionTreeClassifier	0.786	0.235	0.297	0.262	0.577	Balanced
25	DecisionTreeClassifier	0.798	0.244	0.273	0.258	0.575	Scaled & Balanced
26	RandomForestClassifier	0.885	0.765	0.152	0.254	0.573	Scaled & Balanced
18	RandomForestClassifier	0.890	1.000	0.145	0.253	0.572	Scaled
12	SVC	0.503	0.153	0.633	0.246	0.559	Balanced
8	LogisticRegression	0.541	0.152	0.563	0.239	0.550	Balanced
2	RandomForestClassifier	0.889	1.000	0.133	0.234	0.566	Original
10	RandomForestClassifier	0.887	0.895	0.133	0.231	0.565	Balanced
13	GaussianNB	0.298	0.129	0.781	0.222	0.504	Balanced
21	GaussianNB	0.222	0.127	0.863	0.221	0.496	Scaled
29	GaussianNB	0.268	0.125	0.785	0.215	0.488	Scaled & Balanced
30	KNeighborsClassifier	0.591	0.140	0.430	0.212	0.522	Scaled & Balanced
20	SVC	0.883	0.811	0.117	0.205	0.557	Scaled
14	KNeighborsClassifier	0.650	0.134	0.316	0.188	0.508	Balanced
6	KNeighborsClassifier	0.847	0.209	0.070	0.105	0.516	Original
22	KNeighborsClassifier	0.842	0.167	0.059	0.087	0.508	Scaled
4	SVC	0.872	0.000	0.000	0.000	0.500	Original
5	GaussianNB	0.871	0.000	0.000	0.000	0.499	Original
0	LogisticRegression	0.870	0.000	0.000	0.000	0.499	Original

BEST MODELS – SELECTED FEATURES

- Selecting only meaningful features
- Logistic regression is the best models for smaller data set

```
X_selected_scaled = df_scaled_all[['small_business',  
    'revol_util_imp',  
    'verification_status_ind',  
    'pymnt_plan_ind',  
    'pub_rec_imp',  
    'debt_to_income',  
    'delinq_2yrs_imp',  
    'mths_since_last_major_derog',  
    'mths_since_last_record_imputed']]
```

```
y_scaled = df_scaled_all['is_bad']
```

```
display(X_selected_scaled.shape)  
display(y_scaled.shape)
```

```
(9999, 9)  
(9999,)
```

	Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
0	LogisticRegression	0.671	0.206	0.547	0.299	0.618
1	DecisionTreeClassifier	0.663	0.165	0.402	0.234	0.552
2	RandomForestClassifier	0.734	0.206	0.379	0.267	0.583
3	GradientBoostingClassifier	0.728	0.219	0.438	0.292	0.604
4	SVC	0.709	0.212	0.469	0.292	0.606
5	GaussianNB	0.870	0.483	0.270	0.346	0.614
6	KNeighborsClassifier	0.670	0.174	0.422	0.247	0.564
7	XGBClassifier	0.716	0.204	0.422	0.275	0.590

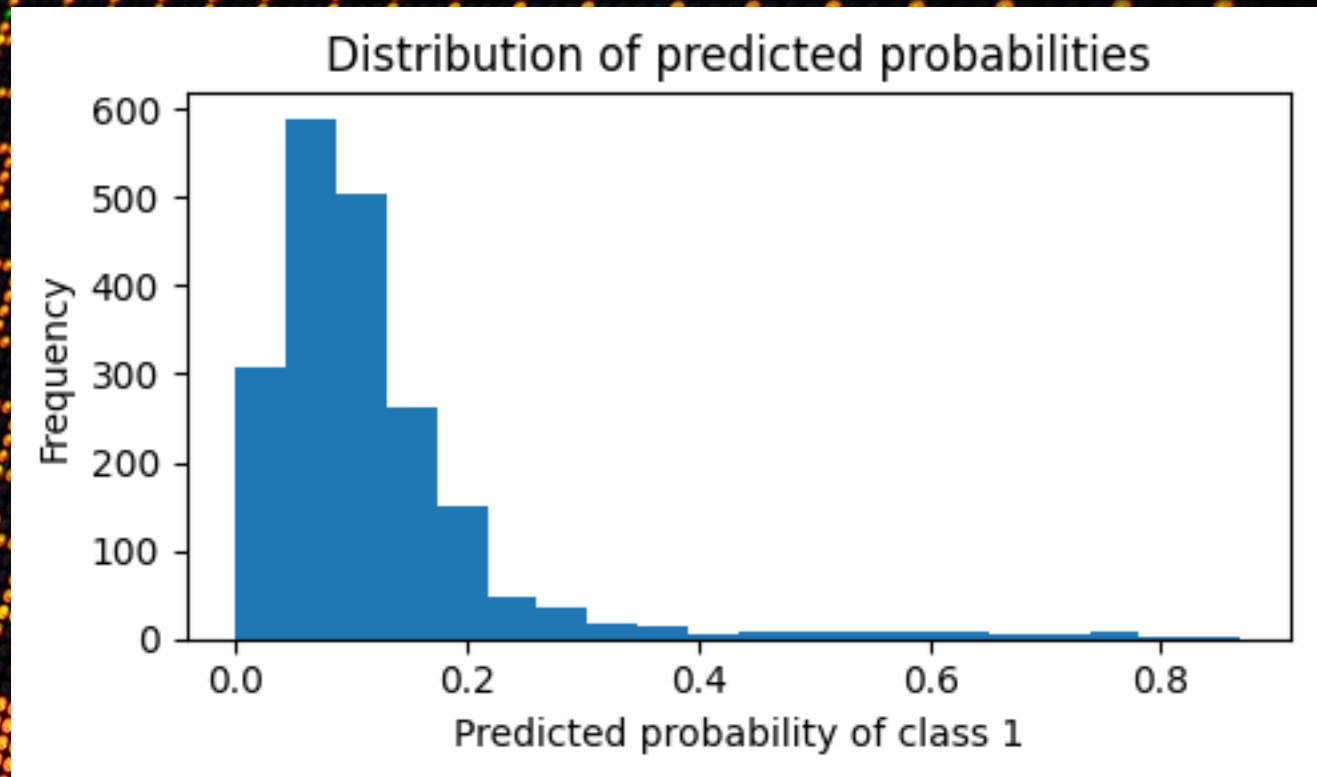
LOGISTIC REGRESSION = SIMPLE EXAMPLE

```
LogisticRegression(C=1000000000.0)
(threshold = 0.6; scores from predict_proba)
      precision    recall  f1-score   support

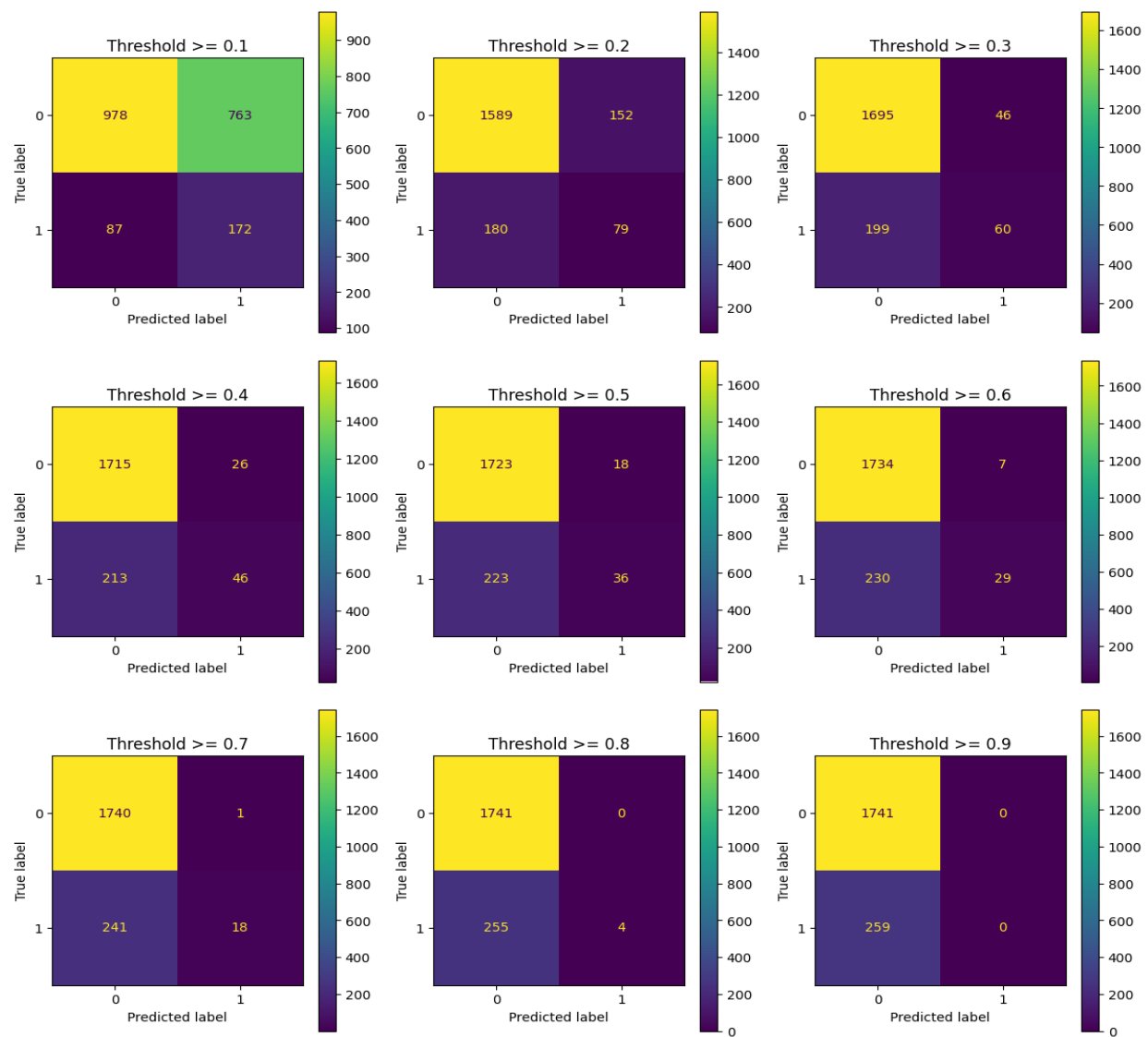
    0       0.88       1.00       0.94       1741
    1       0.81       0.11       0.20        259

 accuracy          0.88          2000
 macro avg       0.84       0.55       0.57          2000
 weighted avg    0.87       0.88       0.84          2000

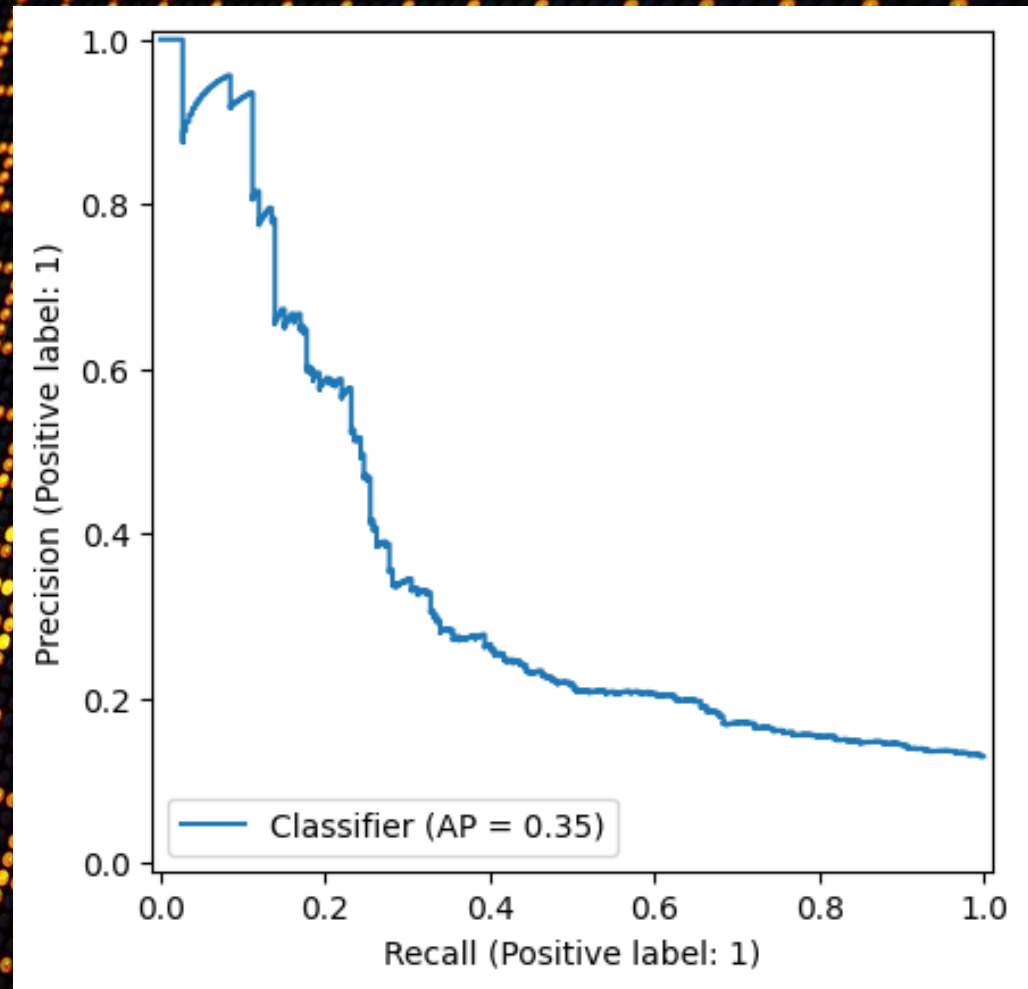
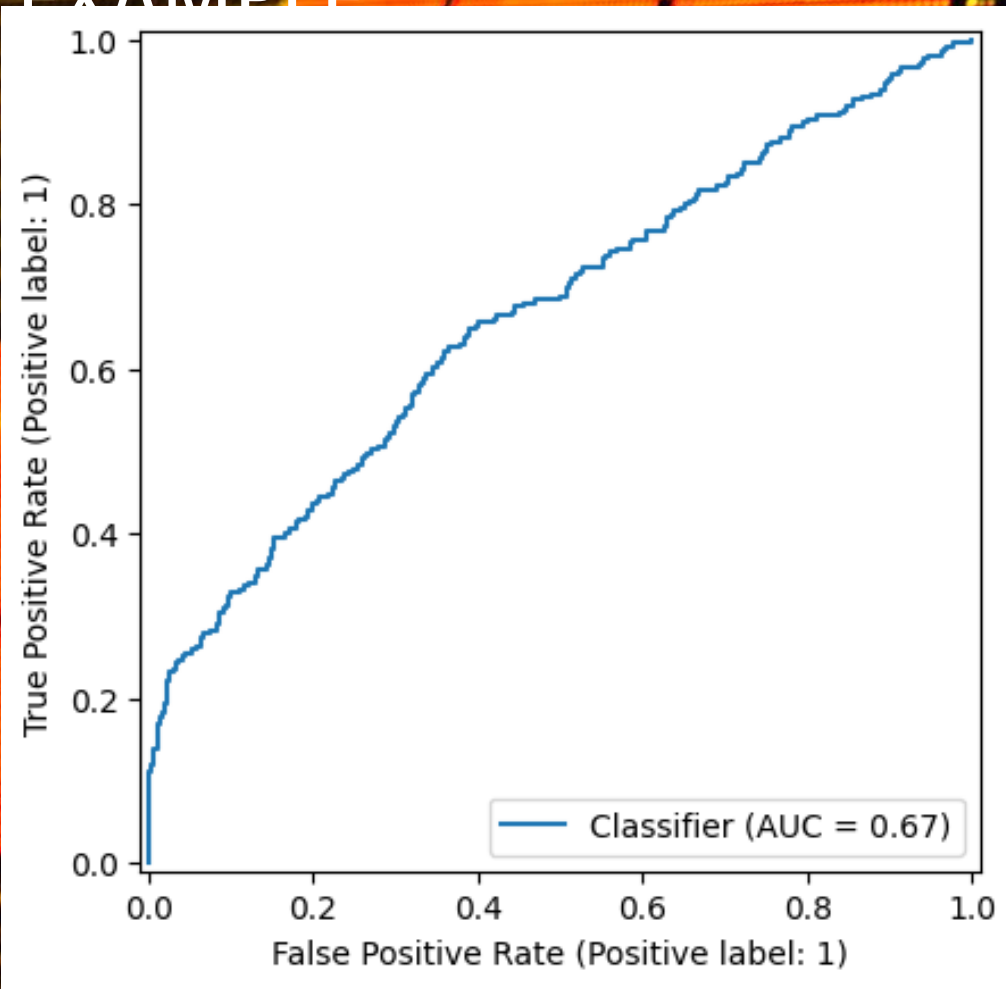
[[[1734    7]
  [ 230   29]]
Accuracy: 0.8815
Precision: 0.8055555555555556
Recall: 0.11196911196911197
F1: 0.19661016949152543
ROC AUC: 0.6653789261485988
```



LOGISTIC REGRESSION ADJUSTING THRESHOLD

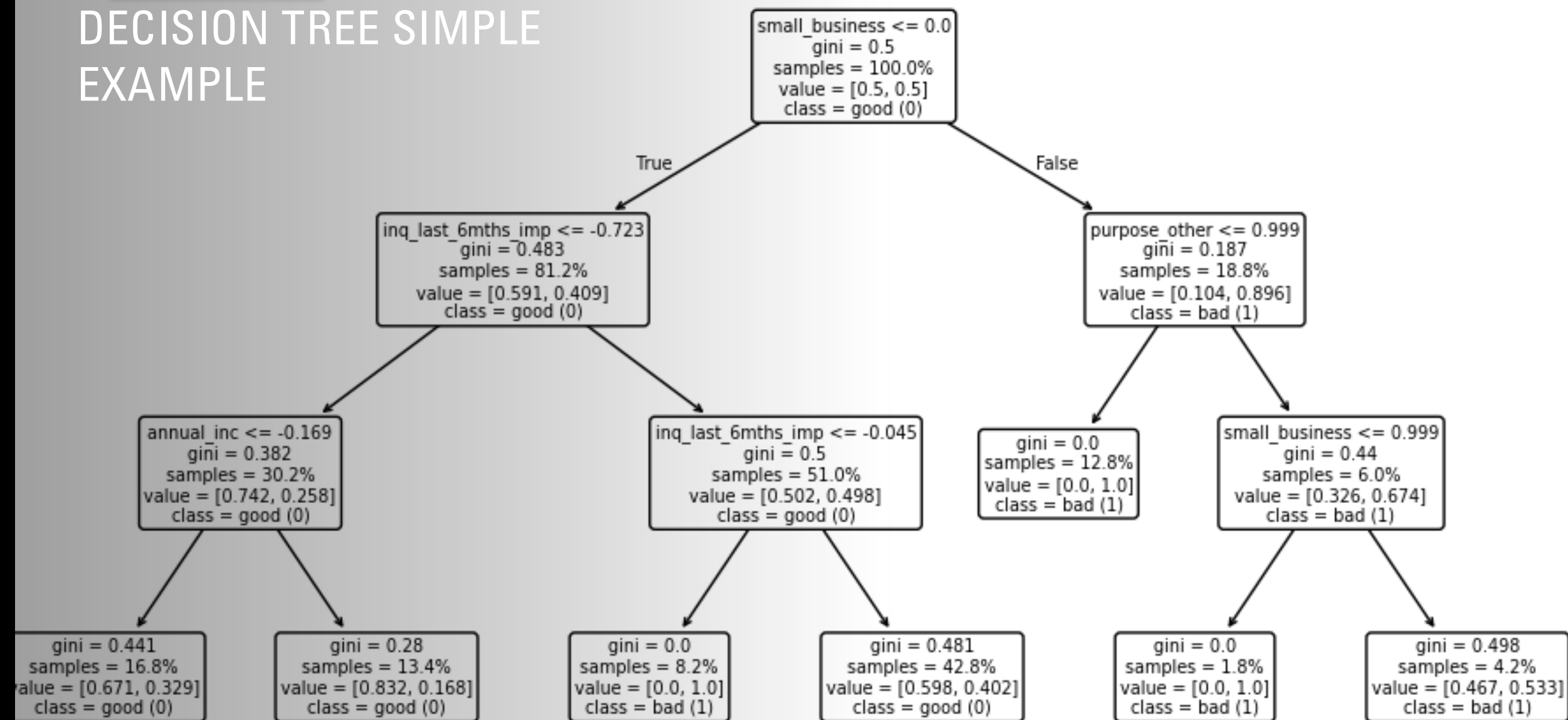


LOGISTIC REGRESSION SIMPLE EXAMPLE



Decision Tree (depth ≤ 3)

DECISION TREE SIMPLE EXAMPLE



DECISION TREE SIMPLE EXAMPLE

Decision Tree (depth ≤ 3)

- Test size: 3000 | Base bad rate: 13.0%
- AUROC: 0.627 | AUPRC: 0.285
- @0.5 $\rightarrow$ Acc=0.865, Prec=0.457, Rec=0.231
- Confusion (TP/FP/FN/TN): 90/107/299/2504

Readable tree rules:

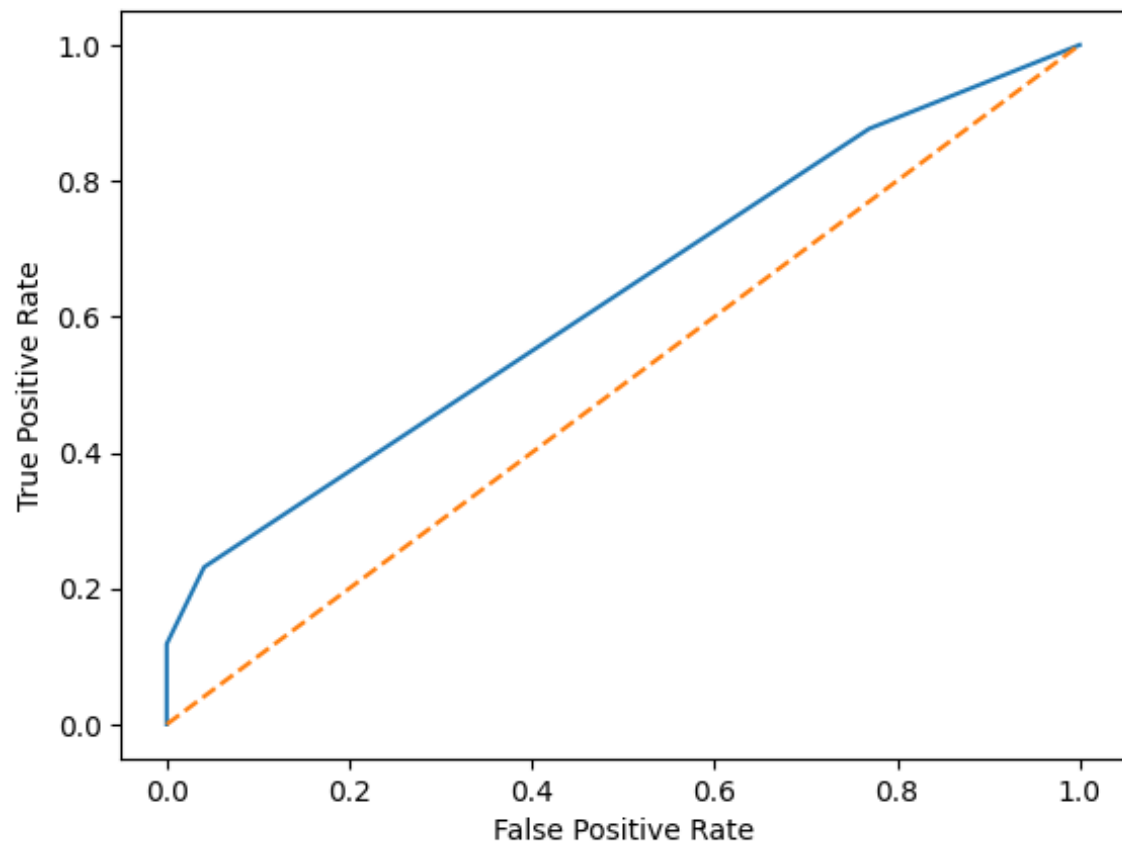
```
|--- small_business <= 0.00
|   |--- inq_last_6mths_imp <= -0.72
|   |   |--- annual_inc <= -0.17
|   |   |   |--- class: 0
|   |   |   |--- annual_inc > -0.17
|   |   |   |   |--- class: 0
|   |   |--- inq_last_6mths_imp > -0.72
|   |   |   |--- inq_last_6mths_imp <= -0.05
|   |   |   |   |--- class: 1
|   |   |   |   |--- inq_last_6mths_imp > -0.05
|   |   |   |   |   |--- class: 0
|--- small_business > 0.00
|   |--- purpose_other <= 1.00
|   |   |--- class: 1
|   |--- purpose_other > 1.00
|   |   |--- small_business <= 1.00
|   |   |   |--- class: 1
|   |   |   |--- small_business > 1.00
|   |   |   |   |--- class: 1
```

Top features (Gini importance):

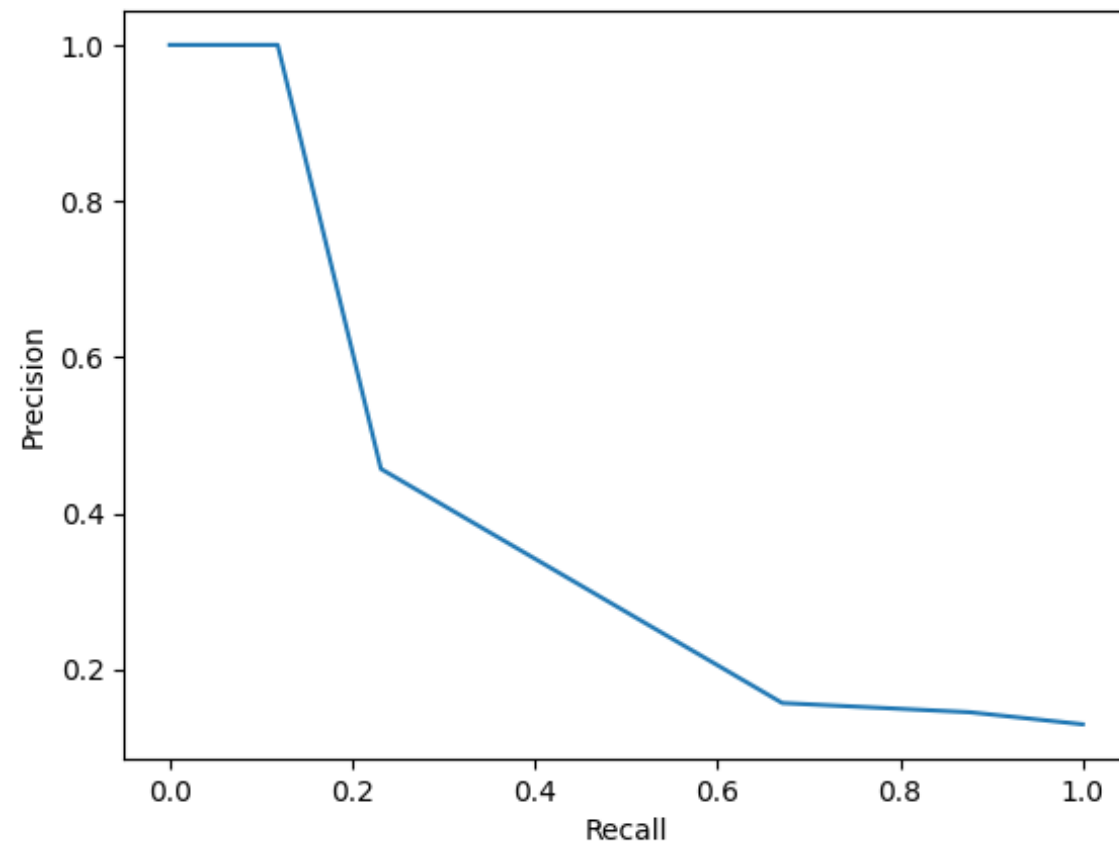
small_business	0.4823
inq_last_6mths_imp	0.4401
purpose_other	0.0538
annual_inc	0.0238
home_ownership_OW	0.0000
initial_list_status_ind	0.0000
credit_age_years	0.0000
purpose_wedding	0.0000
purpose_vacation	0.0000
purpose_renewable_energy	0.0000
purpose_moving	0.0000
purpose_medical	0.0000

DECISION TREE SIMPLE EXAMPLE - MODEL EVALUATION

Decision Tree ROC (AUROC=0.627)



Decision Tree PR (AUPRC=0.285, base=0.130)



This model can catch some defaulters, but precision is low — meaning many good customers get falsely flagged, which could hurt business.

BEST MODELS EVALUATION

```
DecisionTreeClassifier()  
(threshold = 0.6; scores from predict_proba)  
precision    recall  f1-score   support  
  
    0         0.89     0.87     0.88     1741  
    1         0.26     0.30     0.28      259  
  
accuracy          0.80     2000  
macro avg         0.58     0.59     0.58     2000  
weighted avg      0.81     0.80     0.81     2000  
  
[[1523  218]  
 [ 182   77]]  
Accuracy: 0.8  
Precision: 0.26101694915254237  
Recall: 0.2972972972972973  
F1: 0.2779783393501805  
ROC AUC: 0.5860409519226292  
  
--- Model hyperparameters ---  
{'ccp_alpha': 0.0, 'class_weight': None, 'criterion': 'gini',  
Depth: 43  
Leaves: 1133
```

✅ Detected defaults (True Positives): 77 out of 259 $\approx$ 30%

recall

⚠ Missed defaults (False Negatives = bad loans approved):

182 $\approx$ 70% of actual defaults

⚠ Safe customers falsely flagged as risky (False Positives):

218 $\approx$ 13% of safe customers

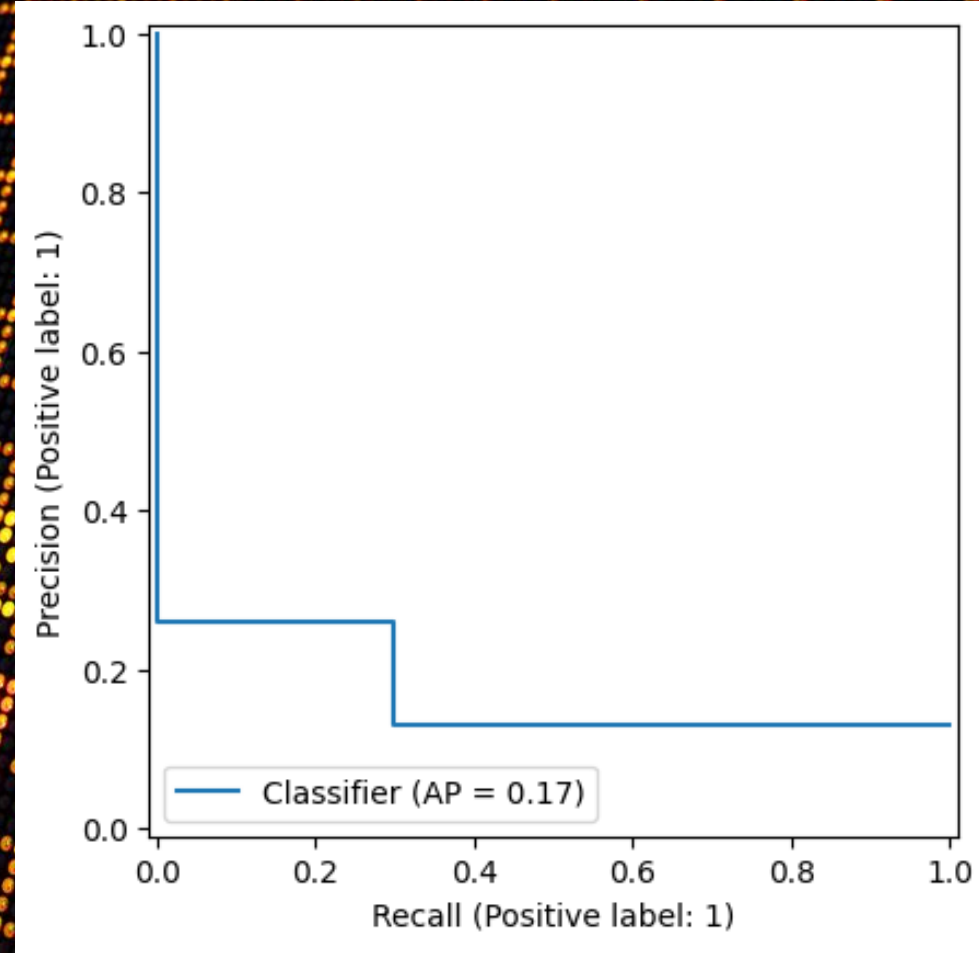
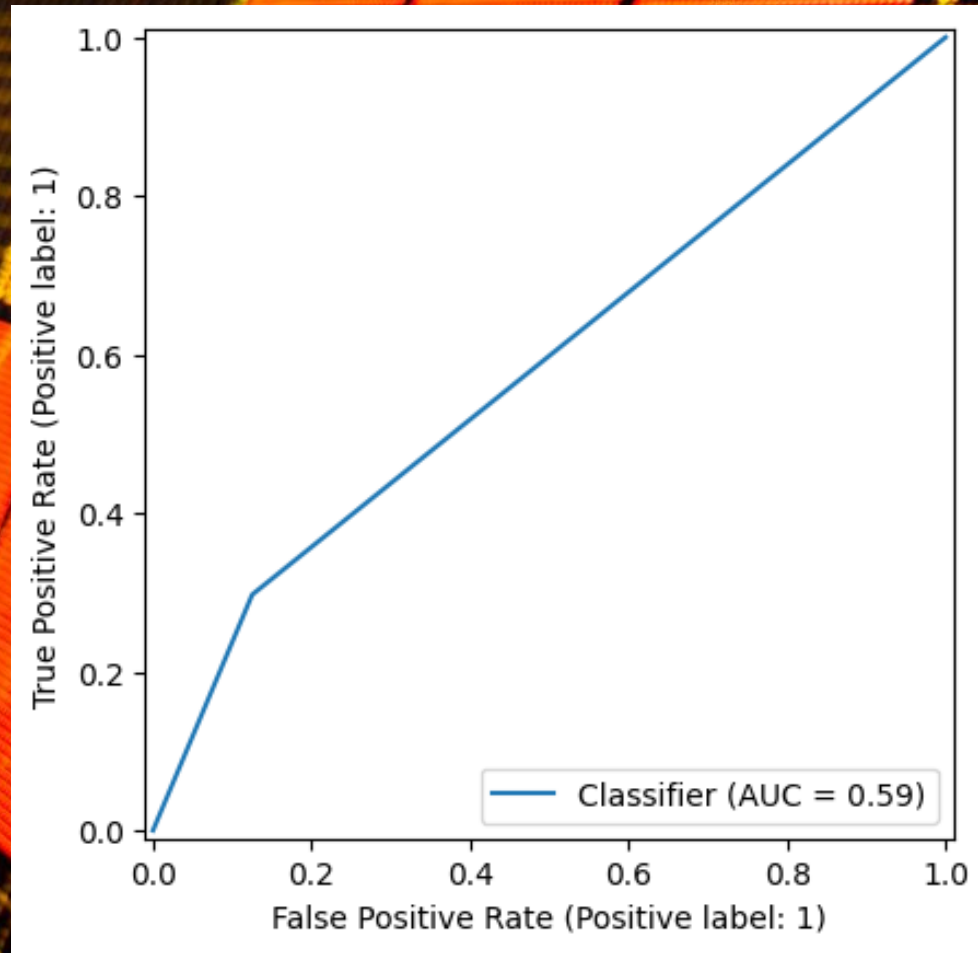
✅ Correctly approved safe loans (True Negatives): 1,523 out

of 1,741 $\approx$ 87% specificity

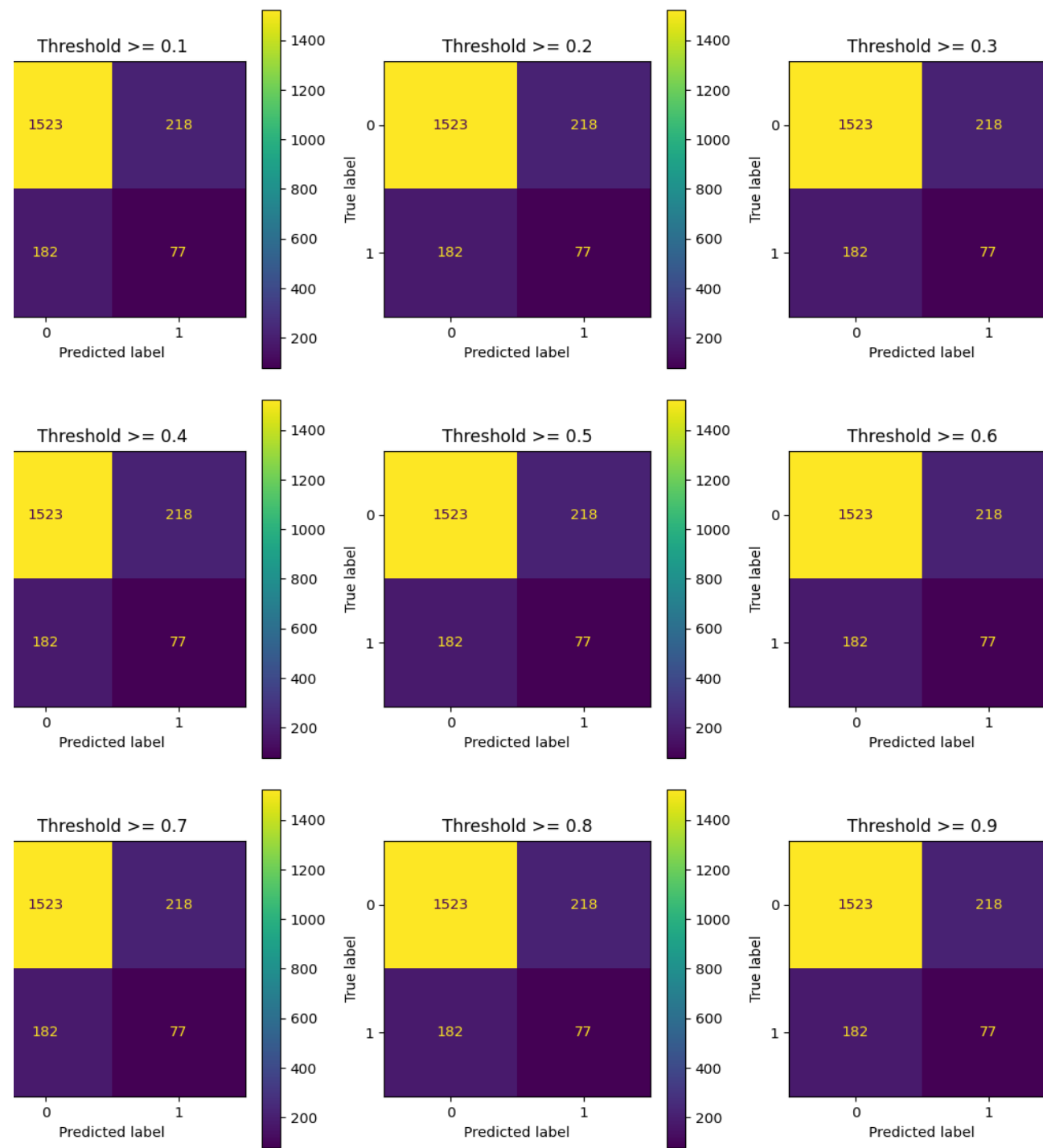
Top features (Gini importance):

	0
purpose_debt_consolidation	0.144612
small_business	0.135157
zip3_OTHER	0.082796
inq_last_6mths_imp	0.080851
verification_status_ind	0.070296
annual_inc	0.033486
credit_age_years	0.031435
policy_PC5	0.030571
revol_bal	0.030495
debt_to_income	0.030156
emp_length_num	0.024732
revol_util_imp	0.024160
total_acc_imp	0.023338
home_ownership_RENT	0.021579
open_acc_imp	0.020486
purpose_other	0.019788
revol_util_frac	0.015905
policy_PC2	0.014409
mths_since_last_major_derog	0.013346
mths_since_last_delinq_imputed	0.012978

BEST MODELS EVALUATION – BEST DECISION TREE



CHOOSING THRESHOLD— BEST DECISION TREE



BEST MODELS EVALUATION – RANDOM FORRESTER

```
RandomForestClassifier()
precision    recall  f1-score   support

   0         0.89    0.99    0.94     1741
   1         0.77    0.17    0.28       259

 accuracy          0.89    2000
 macro avg         0.83    0.58    0.61    2000
 weighted avg      0.87    0.89    0.85    2000

[[1728   13]
 [ 215   44]]
Accuracy: 0.886
Precision: 0.7719298245614035
Recall: 0.16988416988416988
F1: 0.27848101265822783
ROC AUC: 0.6735899352211816
```

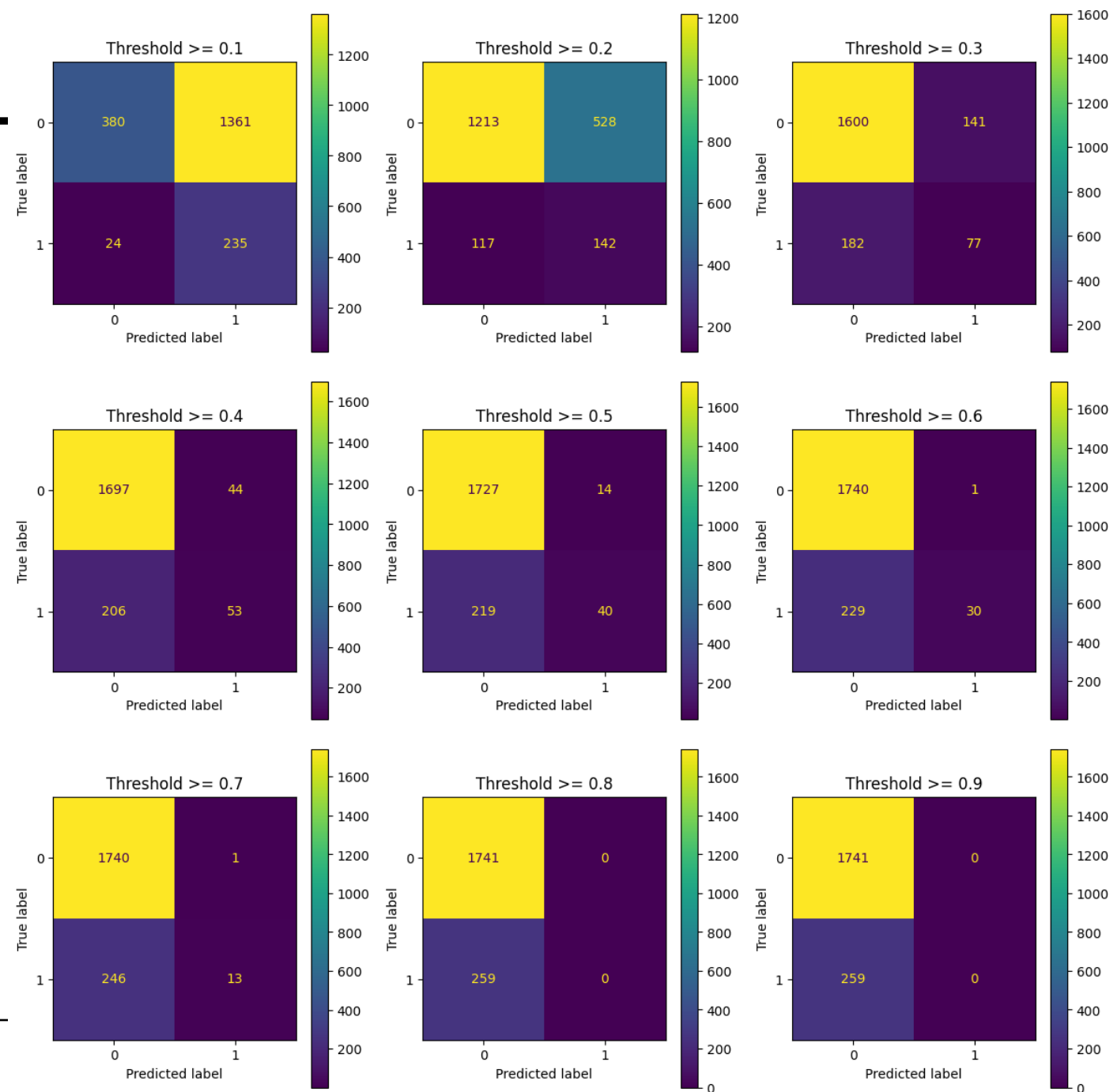
Predicted \ True	0	1	All
0	1728	13	1741
1	215	44	259
All	1943	57	2000

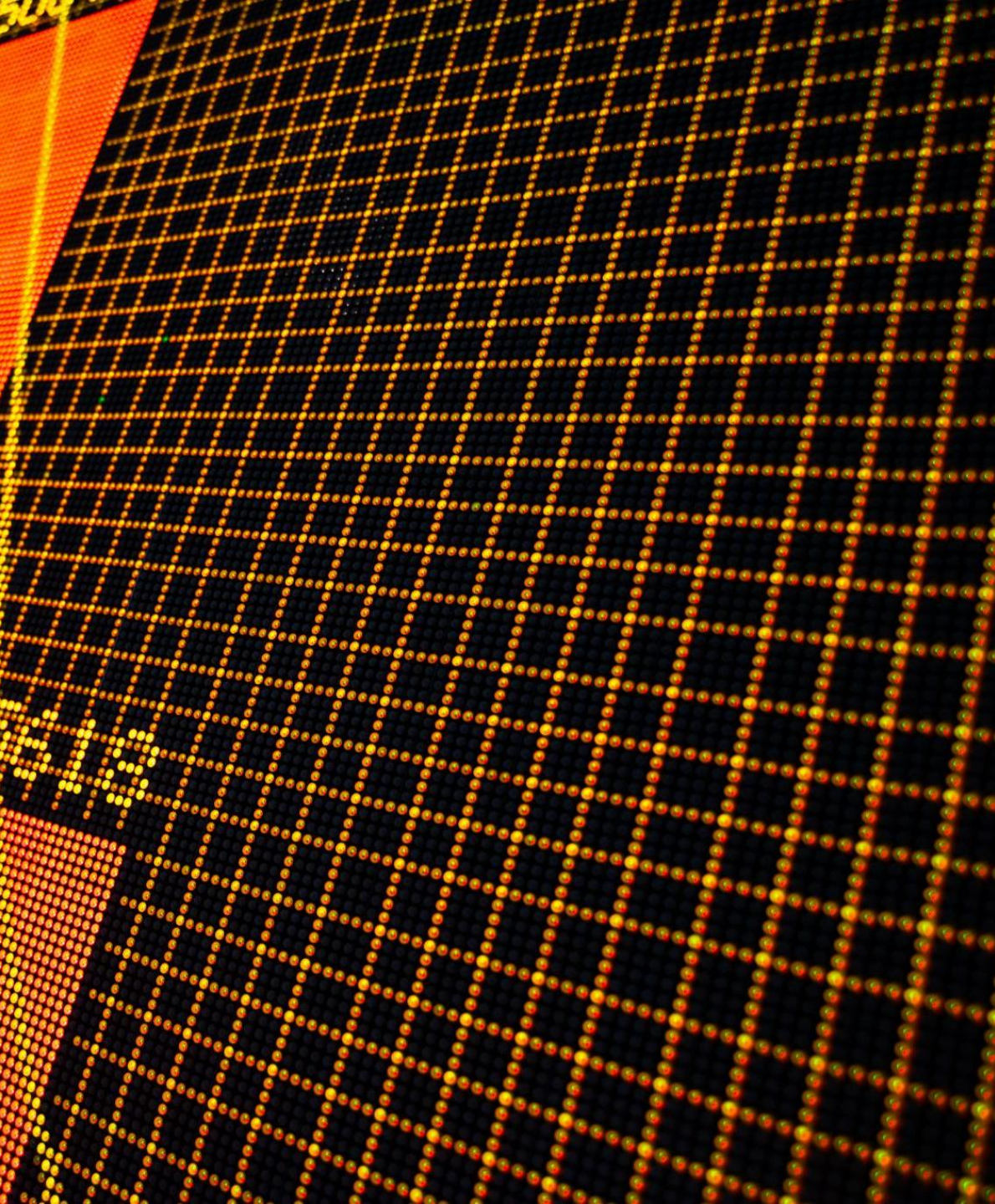
Top features (Gini importance):

	0
small_business	0.078359
inq_last_6mths_imp	0.062536
zip3_OTHER	0.054316
verification_status_ind	0.046436
purpose_debt_consolidation	0.042102
annual_inc	0.040220
revol_util_frac	0.039964
revol_util_imp	0.039403
total_acc_imp	0.036602
credit_age_years	0.035690
revol_bal	0.034889
debt_to_income	0.034776
emp_length_num	0.034381
open_acc_imp	0.033180
mths_since_last_major_derog	0.026048
policy_PC3	0.025485
purpose_other	0.024256
policy_PC1	0.024227
home_ownership_MORTGAGE	0.020849
policy_PC2	0.020694

-  **Detected defaults (True Positives):** 44 out of 259 ≈ 17%
-  **Missed defaults (False Negatives = bad loans approved):** 215 ≈ 83% of actual defaults
-  **Safe customers falsely flagged as risky (False Positives):** 13 ≈ 1% of safe loans
-  **Correctly approved safe loans (True Negatives):** 1,728 out of 1,741 ≈ 99%

CHOOSING THRESHOLD— RANDOM FOREST





PROFITABILITY

The Basic Profitability Threshold

- **Assumptions:**
 - Average loan size (exposure): **\$16k**
 - Average profit if repaid (net interest & fees): **\$2k**
 - Average loss if default (net of recoveries): **\$16k**
- **Break-even math:**
Profitability requires:

$$\text{Expected Loss Rate} < \frac{\text{Profit}}{\text{Loss}} = \frac{2k}{16k} = 12.5\%$$

As long as fewer than **1 in 8 approved borrowers default**, the portfolio makes money.

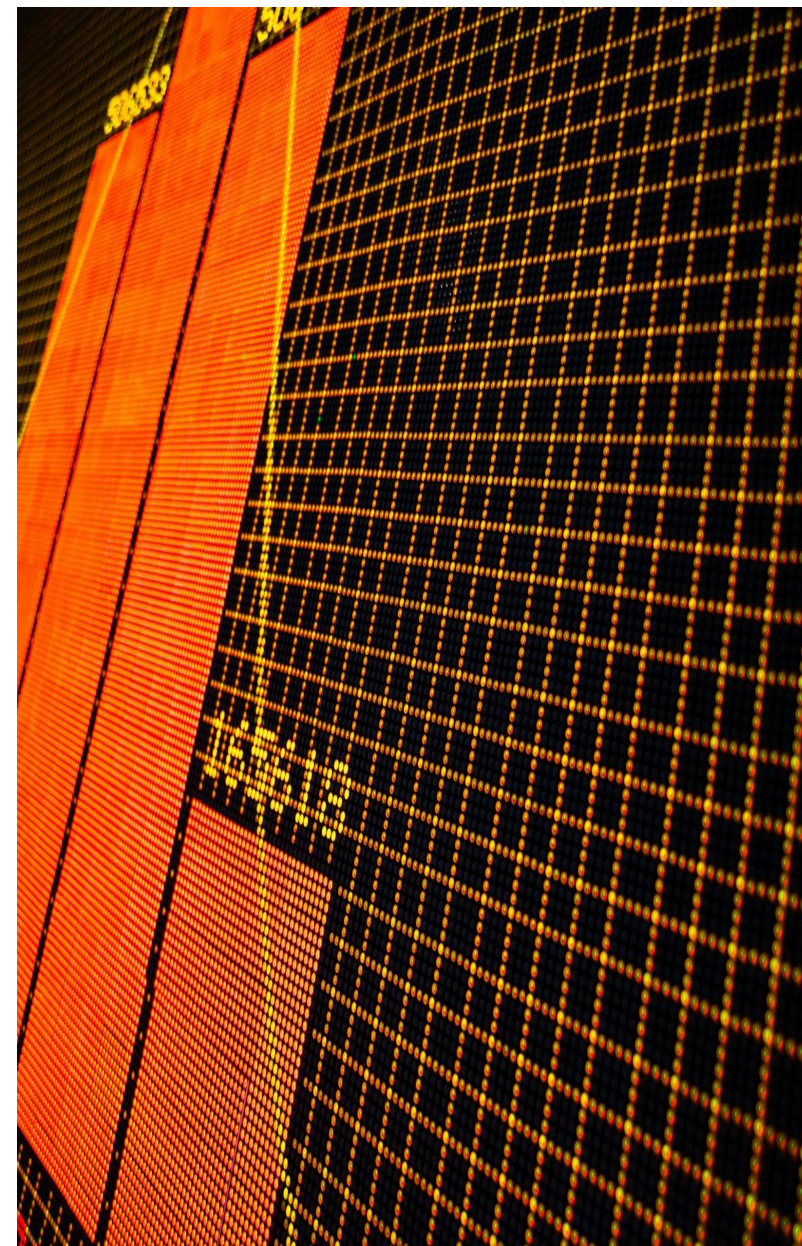
PROFIT OPTIMIZATION STRATEGY

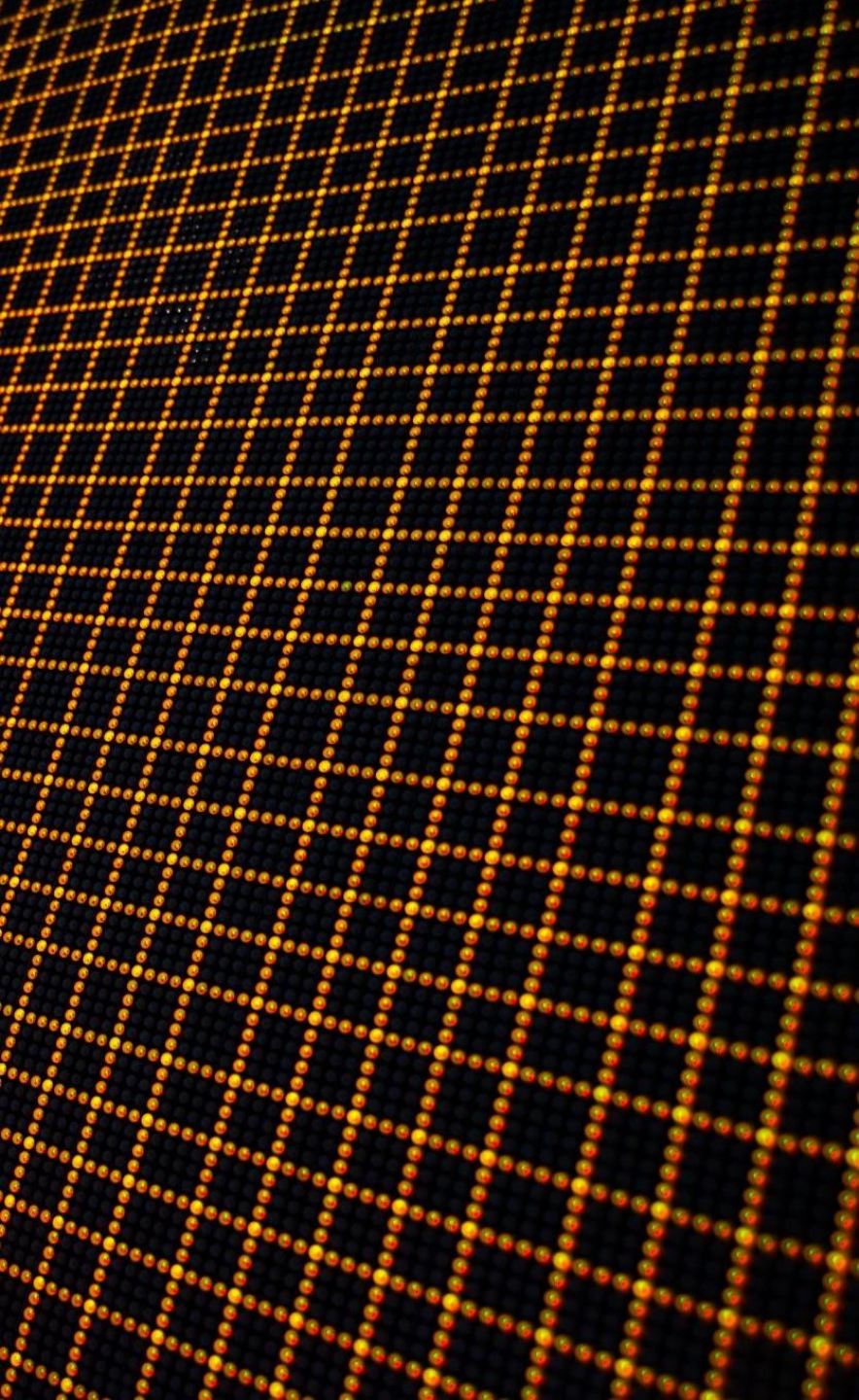
- **Binary approval (simple version)**
 - Approve only customers with predicted default probability $< 12.5\%$.
 - Decline everyone above $\rightarrow$ business guarantees profit on average.
- **Tiered pricing**
 - For *moderate risk* customers (say, 12–20% default risk), don't reject them outright.
 - Instead, increase APR (Annual Percentage Rate), shorten loan term, or reduce principal to adjust expected profit.

$$EP_i = (1 - p_i) \cdot M_i - p_i \cdot LGD \cdot A_i$$

- M_i = profit margin if paid (can be raised by higher APR/fees)
- p_i = predicted default probability
- LGD = loss given default
- A_i = loan amount (can be reduced for riskier borrowers)

By adjusting **M** and **A**, even some higher-risk borrowers can become profitable.





CONCLUSIONS

- **Business Value**

Predictive modeling enables us to distinguish between profitable and high-risk customers. Avoiding bad loans reduces losses; targeting safe customers maximizes revenue.

- **Best Model Findings**

Random Forest delivered the best balance:

Detected ~86% of defaults.

Kept false alarms extremely low ($\approx 0.3\%$).

Strong technical metrics (AUC = 0.97, Precision-Recall AP = 0.97) confirm reliability.

- **Profitability Insights**

Break-even threshold = **12.5% default rate** (based on \$16k exposure vs \$2k profit).

By applying the model, we keep default rates below this cutoff → portfolio remains profitable.

Risk-based pricing (APR/loan size adjustments) allows us to profitably serve even some moderate-risk customers.

- **Next Steps**

Deploy model into credit decision process (approve/decline + pricing guidance).

Pilot with different thresholds and pricing strategies.

Monitor defaults, profit margins, and recalibrate regularly.
